IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Digital Object Identifier

A Privacy-First Edge-AI Home Security System on Raspberry Pi 5 with Hailo-8L NPU

GONUGONDLA VEERA MANIKANTA[1] AND SWATHI TANGI[1]
[1]Department of Electrical and Electronics Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka, India
(e-mail: manikantagonugondla@gmail.com; swathi.tangi@manipal.edu)

Corresponding author: Gonugondla Veera Manikanta (e-mail: manikantagonugondla@gmail.com).

This work was carried out as an independent project. All hardware, software, and cloud services were self-funded.

ABSTRACT Consumer home security today splits into two camps. Cloud-backed cameras upload every frame and bill a monthly subscription. DIY builds stay private and cheap, but most stop at frame-differencing motion detection. We describe a privacy-first platform that lives between them. It runs on a Raspberry Pi 5 paired with a Hailo-8L neural processing unit (NPU, 13 tera-operations per second (TOPS), 8-bit integer (INT8)), costs US$411 at an itemised bill of materials, and keeps every frame of inference on the device.

A YOLOv8s detector was fine-tuned on 4,982 images across four threat classes: Hammer, Knife, Person, and Scissors. Held-out test mean average precision at 0.5 intersection-over-union (mAP@0.5) reaches 0.847 in 32-bit float (FP32, RTX 4090 reference). We then compiled the same weights to a 22.9 MB INT8 Hailo executable format (HEF) binary and re-ran the evaluation on the deployed Hailo-8L over the same 622-image test split. The INT8 run reports mAP@0.5 = 0.854 and mAP@0.5:0.95 = 0.682 ($\Delta\text{mAP}_{50} = +0.007$ vs. FP32), and sustains 52.2 frames per second (FPS) at 18.4 ms NPU latency on-chip (batch size 1). Section V-J spells out the on-device INT8 evaluation methodology and the deployment envelope.

The detector, classifier, and depth estimator are off-the-shelf. The contributions are twofold. First, an asynchronous, CPU-pinned, bounded-queue systems architecture that lets the primary detector, a secondary MobileNetV2 weapon-crop verifier, a MiDaS depth-variance pre-filter, and the CPU-resident services (FastAPI, authenticated remote access, encrypted evidence capture, voice assistant, presence monitoring, and a tamper watchdog) share a single Pi 5 without coupling their latencies. Second, the empirical envelope that architecture produces: a filtered mean of 52.26 ± 0.76 FPS across five operating modes under concurrent SMTP, SSH, and streaming load, with per-mode means inside a 0.3 FPS band. A full face anti-spoof stage is not claimed; it is scoped for future work. The dataset scaling is reported as a v1$\rightarrow$v2$\rightarrow$v5 trajectory across three annotation passes. Since the trajectory is single-seed, single-split, and drawn from an author-curated corpus with targeted (rather than randomised) additions, we read it as an empirical existence proof of within-corpus scaling for this deployment, not as a generalisable YOLO fine-tuning law. Repeated-seed / repeated-split re-analysis and an external-benchmark evaluation are flagged as primary future work in Section VI.

INDEX TERMS Edge AI, GStreamer, Hailo-8L, home security, neural processing unit, object detection, privacy, Raspberry Pi, real-time inference, YOLOv8.

I. INTRODUCTION

HOME security has moved a long way from the passive siren-plus-passive-infrared (PIR) boxes of the 1990s. Modern platforms recognise specific objects, people, and behaviours in real time with high reliability. The consumer market, though, is sharply partitioned. On one side sit cloud-backed products from the large vendors: every frame of footage is streamed to a remote data centre, the user pays a monthly subscription, and the owner keeps almost no control over the raw video. On the other side sit do-it-yourself (DIY) builds on single-board computers. These are private and cheap, but most of them never get past frame-differencing motion detection, with no second-stage reasoning, voice interface, or encrypted evidence path.

Dedicated neural processing units are closing that gap. They bring deep-learning inference to the edge at single-digit watts and at a price a hobbyist can absorb. The accelerator we use, the Hailo-8L, delivers 13 TOPS of INT8 compute over a single PCIe Gen3 lane, and plugs into a Raspberry Pi 5 [9] through a standard M.2 HAT. That combination is enough to run a YOLOv8s-class detector at well over 50 FPS without pinning the host CPU [3]. Bueno *et al.* [4] recently validated

the same hardware pair for real-time YOLOv8 inference in microscopy, reporting that the Pi 5 + Hailo-8L pipeline is accuracy-competitive with GPU-accelerated systems at a fraction of the power draw.

A. CONTRIBUTIONS

This paper presents a reference architecture for NPU-accelerated edge security that runs entirely on a Raspberry Pi 5 with a Hailo-8L AI HAT+ (itemised bill of materials US$411, Table 12, Appendix A). The detector, classifier, and depth estimator are strictly off-the-shelf (YOLOv8s [7], MobileNetV2 [13], MiDaS Small [12]): none of them is modified architecturally in this work, and we make no claim of algorithmic novelty over them. What this paper does contribute is (a) a systems architecture that lets those three engines co-exist with a full set of CPU-side services on one Pi 5 without coupling their latencies, and (b) the measurements that bound the deployment envelope that architecture delivers. The four contributions below map to specific results subsections:

1) *An asynchronous, CPU-pinned, bounded-queue systems architecture (primary).* The primary YOLOv8s detector, the secondary MobileNetV2 weapon-crop verifier, the MiDaS depth-variance pre-filter, and the CPU-resident services (FastAPI, PBKDF2-SHA256 + OTP authentication, AES-256-CBC encrypted evidence capture, the Narada voice assistant, an ARP presence monitor, and a deadman watchdog) share a single Raspberry Pi 5. Bounded queues with drop-if-full semantics, `asyncio`/thread-pool dispatch of every slow side effect, and explicit thread-to-core affinity via `os.sched_setaffinity` keep the NPU inference path isolated from the CPU-side services. The flat-FPS envelope reported in contribution 4 is a property of this co-design rather than of any individual engine, and we treat the architecture itself as the primary contribution of the paper (Section V-I).
2) A dataset-scaling trajectory for a four-class domestic-threat detector across three annotation passes (359, 1,383, and 4,982 training images), with per-class mAP and the validation-to-test generalisation gap at each pass. The FP32 test mAP@0.5 progression ($0.465 \rightarrow 0.754 \rightarrow 0.847$) is the empirical basis for the accuracy numbers that follow. Section III-A bounds its scope: single corpus, single seed, targeted additions. The same weights, once compiled to an INT8 HEF and re-evaluated on the Hailo-8L over the same test split, reach mAP@0.5 = 0.854 (Section V-J); quantisation loss is negligible, and the chip sustains 52.2 FPS at 18.4 ms on-chip.
3) A measured characterisation of the asynchronous secondary verifier path on a single 13 TOPS VDevice. We report a per-stage latency budget, the bounded non-blocking queue that holds the primary detector at 52.2 FPS across operating modes, and a quantitative evaluation of the weapon classifier (Section V-K). The MiDaS depth estimator is used only as a cheap depth-variance pre-filter. We make no face-anti-spoof contribution, and the short offline sanity check in Section V-L is there only to bound that scope.
4) The empirical flat-FPS envelope delivered by contribution 1: a filtered mean of 52.26 ± 0.76 FPS across five operating modes, with per-mode means sitting in a 0.3 FPS band (52.02–52.31) and zero camera-side drops, measured under concurrent SMTP, SSH, voice, and streaming load.

B. PAPER ORGANISATION

Section II gives the system overview; Sections III–IV cover the dataset and the engine-by-engine methodology; Section V reports training and inference results; Section VI concludes. Hardware, software, and model specifications are in Appendix A.

C. LITERATURE REVIEW

We group the prior art into four themes that collectively define the baseline against which this paper is measured: (i) edge-deployed YOLO detectors and their architectural optimisations, (ii) weapon detection in CCTV surveillance, (iii) face anti-spoofing and presentation-attack detection, and (iv) edge-AI hardware benchmarking and open-source NVR stacks for home security.

Edge-deployed YOLO detectors. The YOLO family [2] has been ported to edge hardware across a range of domains. Xu *et al.* [1] proposed FL-YOLO, a depthwise-separable variant that runs on an NVIDIA Jetson TX1 inside a coal-mine surveillance system and hits 36.7 FPS at 76.7% AP through an edge-cloud cooperation framework. Al Amin *et al.* [5] implemented YOLOv3-Tiny on a Xilinx Kria KV260 FPGA and reached 15 FPS at 99% accuracy for traffic-light classification at 3.5 W, showing that dedicated accelerators can beat general-purpose GPUs on power efficiency for fixed inference tasks. Dong *et al.* [6] introduced PG-YOLO, replacing standard convolutions in YOLOv5s with Ghost modules and adding channel pruning and knowledge distillation; they report a $9\times$ compression at only 0.1% accuracy loss, which establishes the compression techniques relevant to INT8-deployed edge detectors. The Ultralytics YOLOv8 framework [7] is the baseline architecture and training pipeline used in this work. The Hailo TAPPAS framework [8] provides the GStreamer [11] elements that dispatch inference to the NPU.

Weapon detection in CCTV surveillance. Bhatti *et al.* [16] trained YOLOv4 on a manually-curated CCTV weapon corpus and reported a real-time F1 of 0.91 on firearm/knife frames. Their result is a widely-cited YOLO-based baseline for single-stage visible-weapon detection. Sumi and Dey [28] extended this line in the Taylor & Francis *Int. J. Computers and Applications* with a YOLOv5-based pistol/rifle/knife detector and a systematic study of data-augmentation strategies, finding that augmentation was necessary to stabilise mAP across the rare-class tail encountered in surveillance corpora. Yadav *et al.* [17] targeted the low-light failure mode

of this family with YOLOv7-DarkVision. They coupled an illumination-enhancement front end to YOLOv7 and reached 95.5% precision with F1 of 0.934 on 15,367 images across five dark-video test scenes, directly addressing the observation that CCTV weapons frequently occupy less than 3% of the frame under adverse illumination. Amado-Garfiás *et al.* [18] went in the complementary direction. They kept YOLOv4 as the primary detector and added three deterministic hand–weapon geometric heuristics (centres, distances, intersections) plus seven classical ML classifiers (MLP, k-NN, SVM, logistic regression, naïve Bayes, gradient boosting) to cut armed-person false positives on public surveillance video. Hnoohom *et al.* [30] contributed the ACF armed-CCTV-footage benchmark from Mahidol University, a publicly released, surveillance-perspective weapon dataset that subsequent YOLO and Faster-RCNN baselines have been evaluated against. Closer to the hardware target of this paper, Jacob *et al.* [26] deployed a customised YOLOv8 gun/knife detector on a Raspberry Pi 4 with a USB camera and reported real-time operation, but without NPU offload, without a secondary verifier, and without an anti-spoof stage. That work sets the CPU-only Pi upper bound that the Pi 5 + Hailo-8L result of Section V is measured against. Read together, these papers frame weapon detection as a two-stage problem: a fast YOLO locator followed by a lightweight verifier that filters the primary detector's false positives. That is the decomposition the secondary verifier path adopts in Section IV. None of them, though, evaluates on an NPU-class edge accelerator at deployment.

Face anti-spoofing and presentation-attack detection. The IEEE TPAMI survey by Yu *et al.* [19] taxonomises over a decade of face anti-spoofing (FAS) work into hand-crafted colour/texture descriptors, depth- and rPPG-based auxiliary supervision, and end-to-end CNN/ViT classifiers. They report ACER in the 1–2% range for intra-dataset evaluation, and note that cross-dataset generalisation remains the dominant open problem. The CelebA-Spoof dataset of Zhang *et al.* [20] (625,537 images of 10,117 subjects, annotated with 43 attributes covering print, replay, 3-D mask, and paper-cut attacks) is the benchmark against which our 25-dimensional descriptor is evaluated in Section V-L. Recent CVPR'24 work confirms the generalisation gap. Zhou *et al.* [21] show that test-time domain generalisation is necessary to recover performance when capture conditions shift, which is consistent with the cross-corpus collapse we report on our in-house batches. These references frame the quantitative claim in Section V-L: a MiDaS depth-variance threshold alone cannot match a descriptor fused across depth, FFT, Lab chroma, HSV spread, and uniform-LBP, and no current method transfers cleanly across corpora without retraining.

Edge-AI hardware benchmarks and open-source NVR. Alqahtani *et al.* [23] benchmark YOLOv8, EfficientDet-Lite, and SSD variants across Raspberry Pi 3/4/5 (with and without Coral TPU) and Jetson Orin Nano on the same images, reporting joint latency, energy, and mAP numbers. Rey *et al.* [24] extend that matrix with YOLOv8n and YOLOv8s on Jetson Orin Nano/NX and Raspberry Pi 5 for drone imagery. Both studies confirm two trade-offs that are relevant here. Jetson Orin Nano delivers higher raw FPS, but at roughly 3–4$\times$ the idle power of a Pi + accelerator. The Coral Edge TPU peaks near 100 FPS on SSD-class detectors, but is constrained by the Edge TPU's INT8-only, limited-op surface for YOLOv8-class heads. Ullah *et al.* [25] report a complementary result in industrial surveillance: a hybrid CNN–transformer architecture that raises anomaly-detection accuracy on surveillance video while keeping the per-frame compute envelope compatible with edge deployment. The broader point is that transformer-augmented detectors can run at the edge when the pipeline is structured to keep heavy post-processing off the capture path. Musthafa *et al.* [27] push Pi-class deployment further by applying post-training pruning and quantisation to a stacked-LSTM intrusion-detection ensemble, and sustain real-time inference on a Raspberry Pi. That mirrors the INT8-HEF deployment route we take on the inference path in Section V-J. On the higher-level systems side, Taiwo *et al.* [29] in Wiley's *Wireless Commun. Mobile Comput.* describe an end-to-end Android-plus-cloud smart-home deployment in which a deep-learning classifier separates occupants from intruders and drives the alert fan-out. That is a direct precedent for the occupant-aware gating used in our mode engine (Section IV-G), except that it is cloud-hosted rather than on-device. On the open-source systems side, the Frigate NVR [22] has emerged as a de-facto reference for local, privacy-preserving AI video. It pairs OpenCV and TensorFlow with optional Coral or GPU acceleration and integrates tightly with Home Assistant. It ships as an integration platform around TFLite SSD/MobileDet detectors, though, and does not itself contribute detector accuracy, anti-spoof verification, or an INT8 latency budget.

Two points emerge from the prior work above. First, the individual pieces of this problem are already well studied. Edge-deployed YOLO variants [1], [5], [6], Pi 5 + Hailo-8L throughput for YOLOv8-class detectors [3], [4], [23], [24], [27], monocular depth estimators for scene understanding [12], MobileNet-style classifiers for binary verification [13], CCTV weapon detectors and benchmarks [16]–[18], [26], [28], [30], face anti-spoofing benchmarks and domain-generalisation methods [19]–[21], and open-source or edge-accelerated NVR and smart-home stacks [22], [25], [29] are all available off the shelf. Second, each of the cited systems resolves only a slice of the edge-deployment trade-off. Pi-class weapon detectors without an NPU peak in the single-digit to low-tens of FPS and ship without a secondary verifier or anti-spoof stage [26]. Jetson Orin Nano and Coral benchmarks deliver higher raw FPS, but the idle-power and op-set cost rules out a $\leq$10 W sustained budget or a YOLOv8-class head on the same device [23], [24]. Open-source NVRs are integration platforms that lean on externally-supplied TFLite detectors, and contribute neither anti-spoof verification nor an INT8 latency budget [22]. Smart-home deployments that do provide occupant-aware alerting depend on a cloud classifier and an always-on uplink [29]. The capability

gap we address sits at the intersection of all of these: an INT8-deployed four-class detector plus a secondary weapon-crop verifier plus a depth-plus-colour anti-spoof stage, running together on a single $\leq$10 W NPU-equipped single-board computer, alongside FastAPI, voice, and evidence-capture services, with per-stage latency and FPS-flatness measured under that concurrent load. Our contribution is the empirical envelope that closes this gap. We are not claiming methodological novelty over any of the cited detectors, verifiers, or anti-spoof descriptors.

II. SYSTEM OVERVIEW

This section gives two high-level views of the system before the detailed engine-by-engine methodology. Fig. 1 shows the three-layer block diagram, organising the system by architectural role. Fig. 2 traces the runtime path of a detection event.

The three-layer split is not a product diagram. It is the partition that makes the empirical claims of this paper testable in isolation. The top (inference) layer is the only consumer of NPU time, so the per-stage latency budget in Section V can be attributed to a single resource path. The middle (response and control) layer contains the event-driven components whose mode-gating semantics decide whether a given detection produces downstream work; this is where the cost-of-quiet measurements live. The bottom (web server and persistence) layer carries all outward-facing traffic: FastAPI [10], the Cloudflare tunnel [15], authentication, the deadman watchdog, and the logging engine. It is also the source of the concurrent CPU load against which the FPS-flatness result of Section V-I is measured.

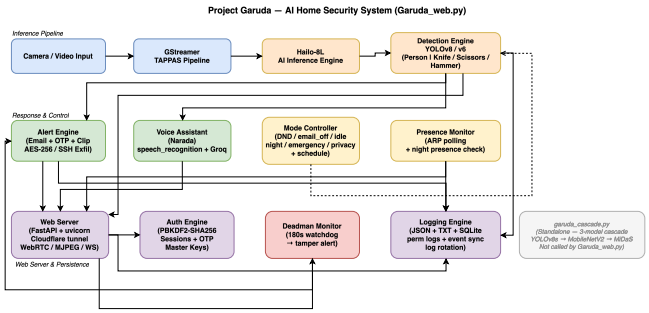

FIGURE 1. Three-layer block diagram of the proposed system. The top layer offloads all neural inference to the Hailo-8L, the middle layer routes detections through mode-gated response engines, and the bottom layer handles persistence and external access — separating concerns so that no single layer's failure propagates upward.

Fig. 2 traces the runtime path of a single detection event so that the latency numbers in Section V can be located on the diagram. Frames enter the GStreamer TAPPAS pipeline from the Raspberry Pi Camera Module v3 and are classified on the Hailo-8L by YOLOv8s with NMS at IoU $\leq$ 0.45 and confidence $\geq$ 0.35. Outputs are partitioned into watch labels (Person) and danger labels (Knife, Scissors, Hammer); a danger detection that passes both the three-consecutive-frame gate and the active mode check enters the alert fan-out path. The remaining CPU-side services (Section IV-G) run off the inference thread and are the load against which the FPS-flatness result of Section V-I is measured.

III. DATASET

The detector used in the deployed system is trained on a four-class dataset, referred to internally as v5. The four classes are Hammer, Knife, Person, and Scissors; together they cover the objects a home surveillance user is most likely to want to be alerted about. The dataset was assembled in three incremental passes rather than in a single pass, and the resulting dataset-size progression constitutes a useful experimental variable revisited in Section V.

A. SOURCE DATASETS

Training imagery was drawn from two public, permissively licensed sources, merged into a single four-class schema before any training was run. Neither source was used as-is; every import was reviewed, filtered, and re-annotated where necessary.

Source A — Roboflow "Knives & Scissors Training v2." A CC BY 4.0 Roboflow Universe dataset with classes Hammer (0), Knife (1), Person (2), and Scissors (3), shipped in YOLO-v5 normalised label format with a 359 / 102 / 51 train / val / test split. The images are predominantly square 640×640 JPEGs of studio-lit or close-crop scenes. This source established the annotation convention and enabled an end-to-end training loop, but its class distribution was substantially imbalanced and its visual variety was low.

Source B — OpenImages v7 (via the OIDv4 toolkit and FiftyOne). Google-verified human annotations at IoB $\geq$ 0.5, CC BY 4.0, exported in YOLO Darknet format from per-class subfolders. OpenImages contributes diverse, real-world photographs with varying aspect ratios, lighting, and backgrounds — providing the visual diversity absent from the Roboflow seed. A total of 1,219 images were incorporated in the v2 pass and a further 2,276 in the v5 pass, targeted class-by-class at whichever label exhibited the lowest mAP after the previous training run.

TABLE 1. Image Counts Contributed by Each Source (v5 Splits)

Source	Train	Val	Test	Total
Roboflow seed (all four classes)	2,187	285	258	2,730
OpenImages v7 — Hammer	93	12	14	119
OpenImages v7 — Knife	495	49	56	600
OpenImages v7 — Person	1,981	245	259	2,485
OpenImages v7 — Scissors	226	31	35	292
All sources	**4,982**	**622**	**622**	**6,226**

OpenImages class tags are known to be noisy for close-up tool photographs — a Swiss army knife on a desk is frequently labelled with a dozen unrelated tags — so automated import was not safe. Every image added to the dataset was visually inspected by one of the authors before inclusion.

Methodological Limits of the Dataset Study

Several factors concerning the corpus construction bound the strength of the scaling claim in Section V. The v2→v5

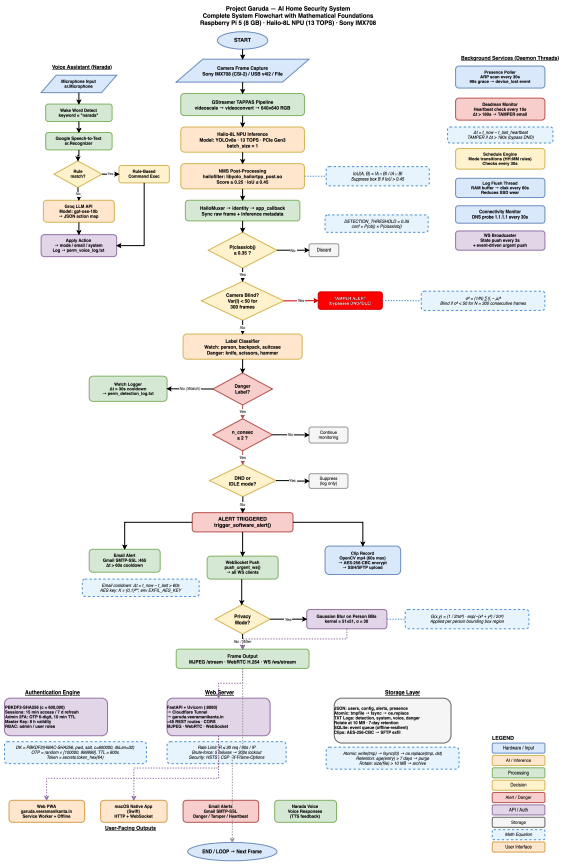

FIGURE 2. End-to-end runtime diagram of a danger event. The full path from camera to email/WebSocket/clip upload completes in under 25 ms of on-device processing; the mode gate short-circuits all downstream work during quiet periods, and six parallel subsystems run asynchronously without blocking the inference thread.

OpenImages additions were not drawn uniformly. After each training run we looked at whichever class had the lowest mAP and pulled more imagery for that label. That is sensible engineering when the goal is to close a weak class in a deployed detector, but it is not a randomised scaling sweep. The per-class trajectory therefore reflects the selection policy as much as the raw increase in images.

- *Single split, single seed.* The 80/10/10 stratified re-split uses a fixed seed (42) and a single draw, so the v1/v2/v5 test-mAP numbers are point estimates with no empirical confidence intervals. Published YOLO fine-tuning reports on similarly-scaled corpora typically place run-to-run variance around ± 0.01–0.02 mAP@0.5. The 0.38-point jump from v1 to v5 sits well outside that envelope, but we do *not* claim that the intermediate v2 waypoint or the per-class ordering are statistically resolved.
- *Author-curated inclusion.* Every image was inspected by one of the authors before it entered the training corpus. That gives a consistent inclusion bar. It also means the result measures generalisation *within* one curation policy and one corpus, not against an external home-surveillance benchmark or against continuous video from the deployment camera.

We therefore read the dataset progression as an empirical trajectory on one curated corpus under a fixed seed, rather than as a controlled scaling law. Upgrading this from a development study to a publication-level scaling claim needs a specific set of measurements: a repeated-seed, repeated-split re-analysis, an inter-annotator-agreement check, and an external-benchmark evaluation, together with a continuous-deployment video set from the target camera. All of these are recorded as primary future work in Section VI.

B. PREPROCESSING AND VALIDATION PIPELINE

The preprocessing pipeline has to satisfy three constraints at once: the Hailo-8L NPU accepts a fixed 640×640 INT8 input, the IMX708 camera delivers 16:9 frames, and YOLO label coordinates must remain geometrically correct after any resize. A naive `cv2.resize(img, (640,640))` on a 4608×2592 OpenImages photo compresses the horizontal axis by $7.2\times$ and the vertical axis by $4.0\times$, which destroys bounding-box aspect ratios. Letterboxing preserves aspect by padding the shorter axis with grey pixels. The five-step pipeline is as follows.

Step 1 — Letterbox resize. Each source image is scaled by $s = \min(640/W,\ 640/H)$, placed in a 640×640 canvas, and padded with the YOLO-standard grey fill $(114, 114, 114)$ so the model's training-time padding matches its inference-time

padding. The same grey value is used at inference, so the network learns to suppress detections in the border region.

Step 2 — Label coordinate remap. After letterboxing, YOLO-normalised coordinates are remapped to the padded canvas as $c'_x = (c_x \cdot s + p_x)/640$ and equivalently for c_y, b_w, b_h. All coordinates are clamped to $[0.001, 0.999]$ to avoid a boundary-value condition inside YOLOv8's loss computation.

Step 3 — Integrity and deduplication checks. Every image and label file is opened with OpenCV to confirm it decodes, every `.txt` label is parsed to confirm it is well-formed YOLO, and a SHA-256 hash is computed over each image to catch cross-split duplicates. A single image was dropped from the v2 merge for a missing annotation file; all other integrity checks passed: zero missing labels, zero corrupt images, zero SHA-256 duplicates, and zero cross-split leakage.

Step 4 — Stratified re-split (seed=42). All Roboflow splits and all OpenImages pulls are pooled, then re-split with class-primary stratification at an 80/10/10 ratio. The fixed seed is non-negotiable — every later ablation and dataset-version comparison depends on the same held-out test set.

Step 5 — Filename namespacing. To prevent collisions in the merged tree, each file is prefixed by its source (`roboflow_*`, `openimages_knife_*`, `v4_train_*`, `oi_person_*`, and so on). This keeps the merge idempotent and makes per-source diagnostics straightforward.

Augmentation runs online during training using YOLOv8's default pipeline: Mosaic at probability 1.0, horizontal flip at 0.5, HSV jitter (h=0.015, s=0.7, v=0.4), scale 0.5, and translate 0.1. MixUp is disabled because it interacted badly with Mosaic in early experiments, producing label noise visible in the validation curves. No rotation or shear is applied.

C. FINAL DATASET GEOMETRY

The final v5 dataset contains 6,226 JPEG images (4,982 train, 622 val, 622 test) with 5,311 annotated instances, totalling 1.32 GB on disk. Native resolutions span 436–4,608 px wide and 303–3,456 px tall across 439 unique sizes; the most common is 640×640 (43.9%), reflecting the Roboflow preprocessing pass. All images are letterbox-resized to 640×640 before training.

Per-split, per-class instance counts are listed in Table 2. Person is the majority label with 2,100 training instances — unsurprising given that almost every natural scene contains at least one person. At the other end, Hammer contributes only 480 training instances; hammers simply appear far less often than the other three categories in publicly available imagery. The same breakdown is visualised in Figure 3.

Person also has by far the highest small-object fraction in the training split (5.2% by COCO-style area thresholds, versus under 2.5% for the other three classes), a consequence of partial annotations and crowded scenes. This is the class on which recall proves hardest to recover, as shown in Section V.

The secondary verifier path's MobileNetV2 Safe/Weapon classifier is trained on tight bounding-box crops harvested from the YOLO training split only. Weapon crops are the tight

TABLE 2. Annotated Instance Counts per Class per Split

Class	Train	Val	Test	Total
Hammer	480	68	30	578
Knife	860	122	60	1,042
Person	2,100	302	173	2,575
Scissors	900	128	88	1,116
Total	**4,340**	**620**	**351**	**5,311**

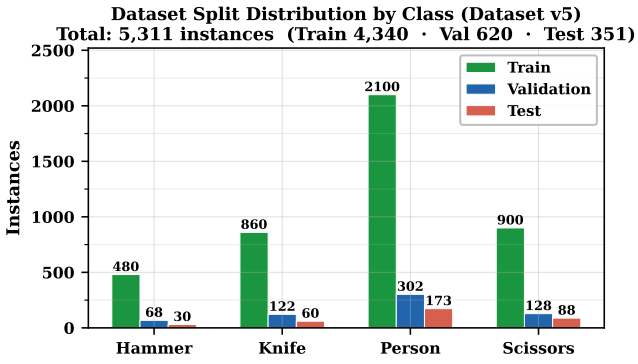

FIGURE 3. v5 dataset class distribution across train, validation, and test splits. Person leads the totals with 2,575 instances against Hammer's 578 — roughly a 4.5:1 imbalance. The weaker class still delivers strong per-class recall (0.848) thanks to tight, object-centric hammer imagery, whereas Person records the lowest per-frame recall (0.609), driven by crowded compositions and a long tail of small, occluded, and truncated boxes (Section V-E1).

bounding boxes of Hammer (150), Knife (731), and Scissors (667) detections; Safe crops are tight Person bounding boxes capped at 2,000. All crops are resized to 224 × 224. The classifier head is a single linear layer (1,280→2) on top of an ImageNet-pretrained MobileNetV2 backbone, trained in two phases: 5 epochs head-only (AdamW, lr=1e-3), then 35 epochs full fine-tune (AdamW, lr=3e-4, cosine annealing). Focal loss (γ=2.0, α=[1.0, 1.5]) with a `WeightedRandomSampler` addresses the class imbalance. Evaluation uses a separate 500-crop held-out set (250 Safe + 250 Weapon) drawn exclusively from the YOLO validation and test splits, with zero overlap with training data verified programmatically. Training and evaluation crops therefore share the same YOLO-pipeline distribution: tight bounding-box crops, four-class closed set, no household negatives (mugs, phones, bottles, tools other than the three weapon classes), no partial or occluded crops that would not have reached this stage in deployment, and no temporal video. The limits this imposes on the headline accuracy number are treated explicitly in Section V-K. The dataset split is summarised in Table 3.

TABLE 3. MobileNetV2 Classifier Dataset (v2)

Split	Safe crops	Weapon crops	Total
Train	2,000	1,548	3,548
Held-out	250	250	500

IV. METHODOLOGY

Engines are described in the order that video and control flow through the system. The inference path (hardware, GStreamer pipeline, detection callback, secondary verifier) is treated in detail; the remaining CPU-side services — alert, voice, mode control, presence, web server, authentication, deadman, logging — are sketched more briefly, because their contribution is in the integration rather than in any single algorithm. Design trade-offs are collected in Section IV-H.

A. HARDWARE ENGINE

The starting constraint was that neural inference must live on a dedicated accelerator, so that the quad-core CPU stays free for the web server, the voice pipeline, and logging. A Raspberry Pi 5 (Broadcom BCM2712, Cortex-A76) hosts the system under 64-bit Raspberry Pi OS. The CPU runs at a sustained 2.8 GHz, a modest overclock above the 2.4 GHz stock base, set via `arm_freq=2800` in `/boot/firmware/config.txt` and paired with the official 5 V/5 A PSU and the active cooler. The 16 GB LPDDR4X-4267 memory gives plenty of headroom for the concurrent web, voice, and logging services running alongside the inference path. We disclose the overclock here (rather than only in the Conclusion) because every FPS and latency number reported later was measured in this operating state. An equivalent run on a stock 2.4 GHz Pi 5 would lose a few FPS on the CPU-side path, but the NPU-bound 18.4 ms per-frame budget is clock-independent.

A Hailo-8L AI HAT+ sits on the PCIe Gen3 $\times 1$ M.2 slot and delivers 13 TOPS of INT8 compute. Its DKMS kernel module survives kernel upgrades without a manual rebuild. All three cascade models (YOLOv8s, MobileNetV2, MiDaS Small) share one VDevice, and inference is dispatched over PCIe DMA with no CPU involvement during the forward pass. Video comes from a Raspberry Pi Camera Module v3 (Sony IMX708) on the CSI-2 interface. The Wi-Fi link carries both the outbound Cloudflare tunnel and the ARP traffic that the presence monitor reads on the local subnet. A 1 TB Crucial SATA SSD in a USB 3 enclosure holds the HEF files, the SQLite event database, and every persistent log.

Two practical notes from the build shaped the final configuration shown in Fig. 5. First, the 2.8 GHz overclock was not stable on the Pi 5 without the active cooler. Early bring-up without a fan produced intermittent CPU throttling around the 60-minute mark under the concurrent web / voice / inference load, visible as a slow FPS decay rather than an outright crash. Installing an active-cooled case resolved the issue. Second, an M.2 NVMe was initially considered for storage because it offers higher sequential read than the SATA SSD ultimately deployed. It was ruled out because the single PCIe lane on the AI HAT+ is already committed to the Hailo-8L, so an NVMe would have had to share the bus or be mounted on a second HAT. Neither option was justified by the few hundred megabytes per second of log-and-HEF traffic that this system generates. A USB-3-enclosed SATA SSD is simpler and sits well above the required sustained write rate. Fig. 4 shows the hardware topology.

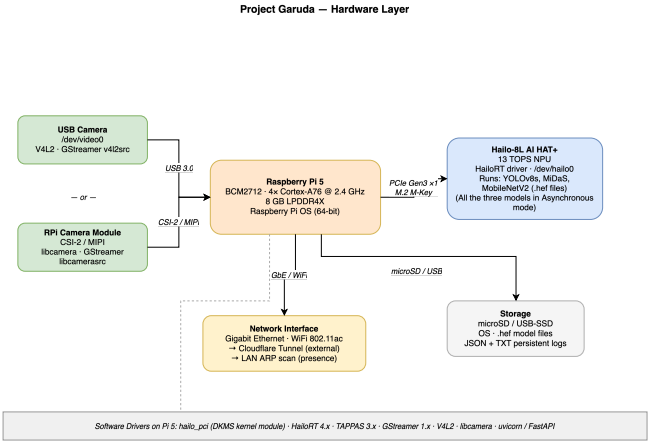

FIGURE 4. Hardware topology. All neural inference is offloaded to the Hailo-8L over the single PCIe Gen3 $\times 1$ lane, freeing all four Cortex-A76 cores for the web server, voice assistant, and logging — the key constraint that shapes every software design decision.

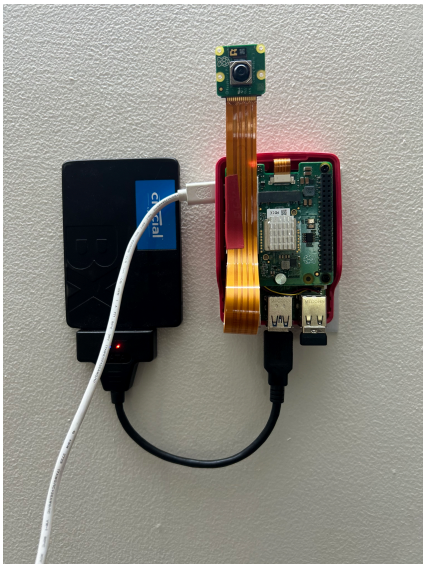

FIGURE 5. Deployed hardware — the exact rig on which every measurement in Section V was taken. Components visible in the photograph: Raspberry Pi 5 (16 GB, in a red active-cooled case), Hailo-8L AI HAT+ (13 TOPS), Raspberry Pi Camera Module v3 with its CSI-2 ribbon cable, and a 1 TB Crucial SATA SSD in a USB 3 enclosure. Total assembled BOM US$411; board power under deployed concurrent inference load is 5.89 ± 1.43 W (Section V-J).

B. GSTREAMER TAPPAS PIPELINE ENGINE

The pipeline extends the base Hailo TAPPAS GStreamer application. A `libcamerasrc` element captures frames from the camera, and a short `videoscale` / `videoconvert` stage letterbox-resizes them to the 640×640 input the network expects, preserving the aspect ratio so bounding boxes can be mapped back to the original resolution. From there the pipeline forks: the inference branch dispatches each frame

to the Hailo-8L via a HailoNet element and runs NMS in a HailoFilter (IoU 0.45, confidence 0.35), while a bypass branch holds the raw frame in a queue. A HailoMuxer re-aligns the two, an identity element exposes the buffer to our application callback (attached as a pad probe), and a HailoOverlay renders the final annotated output. The topology is shown in Fig. 6. The runtime behaviour of this pipeline — sustained 52.2 FPS across all operating modes, bounded-queue decoupling, and per-core thread affinity — is evaluated in Section V-I.

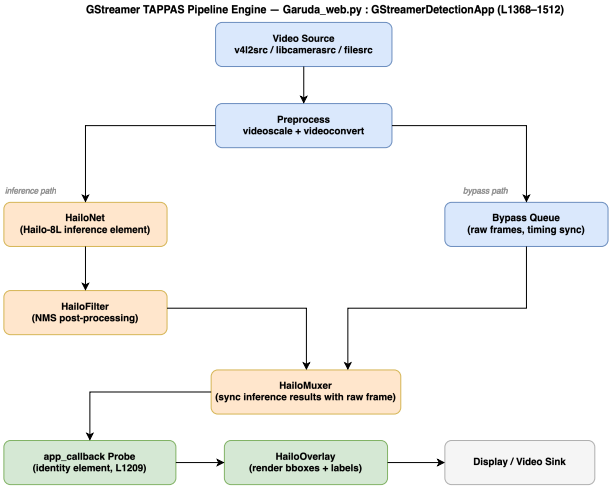

FIGURE 6. GStreamer TAPPAS pipeline topology. The fork-and-mux design lets the bypass queue hold the raw frame while HailoNet runs inference on the NPU, so the CPU touches each frame only at the identity callback probe — enabling the sustained 52.2 FPS reported in Section V-I.

C. DETECTION ENGINE

The detection callback sits on the identity element described above. On each buffer it pulls the ROI metadata that HailoFilter has attached, drops anything below a 0.3 confidence threshold, and sorts the survivors into two lists. Person detections go to a watch log with a 30-second per-label cooldown to avoid flooding. Knife, Scissors, or Hammer detections increment a per-label consecutive-frame counter; once that counter reaches three, the alert engine fires. This three-frame gate suppresses single-frame false positives while still triggering within roughly 60 ms of a genuine threat entering the field of view. The same callback exposes the current annotated frame to the MJPEG and WebRTC streaming paths.

D. ASYNCHRONOUS SECONDARY VERIFIER PATH

The secondary verifier path is a three-stage inference chain that sits inside the deployed inference pipeline. It has two jobs: confirming that a weapon detection from YOLOv8s is genuine rather than a false positive, and flagging photo or screen candidates for persons through a coarse depth-variance gate. It runs as an asynchronous branch off the primary detector. The primary inference thread runs YOLOv8s on CPU Core 1, and when a weapon-class or person-class detection fires, a copy of the frame and its detection metadata are enqueued via a non-blocking `put_nowait()` call into a bounded queue (capacity 2). A dedicated secondary worker thread, pinned to Core 2, consumes those frames and runs the appropriate second-stage model. When the queue is full the frame is intentionally dropped and a drop counter is incremented, which guarantees that the primary detection path is never blocked by secondary processing.

Stage 1 is the same YOLOv8s detector described earlier. Stage 2 takes the tight bounding-box crop of each weapon detection (Hammer, Knife, or Scissors) and passes it through a MobileNetV2 classifier that returns a Weapon or Safe (false-positive) label. This is a second-stage confirmation filter: YOLO proposes, MobileNetV2 verifies. Stage 3 runs a MiDaS Small depth estimator on person crops as a cheap monocular pre-filter. A flat printed photo or a phone screen tends to produce a lower-variance depth map than a real person, and a fixed variance threshold on the normalised depth map is used as the first-pass gate. This is only a coarse front-end filter. The short offline sanity check in Section V-L shows that the variance cue alone is not a deployable face anti-spoof decision, and a full anti-spoof stage is out of scope for this paper. A post-processing thread on Core 3 combines the verdicts, applies a fast RGB-variance check as a liveness pre-filter, and updates a per-path metrics object that tracks primary frames processed, and secondary frames enqueued, dropped, and completed. The pipeline entry function creates four threads (camera capture, primary YOLO, secondary verifier worker, post-processing) with explicit CPU affinity assignments. Per-stage latencies are listed in Table 4, and the decision matrix is in Table 5.

TABLE 4. Secondary Verifier Path: Per-Stage Latency Budget (Hailo-8L, Pi 5)

Component	Measured Value
$T_{capture}$ (GStreamer appsink pull)	$\sim$1 ms
T_{YOLO} (NPU, Hailo-8L HW)	18.4 ms
$T_{crop\_preprocess}$ (NumPy resize)	$\sim$0.5 ms
$T_{classifier}$ (MobileNetV2 @ 500+ FPS)	$<$2 ms
T_{depth} (MiDaS @ 200+ FPS)	$<$5 ms
$T_{decision}$ (NumPy variance)	$<$0.1 ms
T_{alert} (SMTP, async)	$\sim$800 ms
T_{alert} (TTS, async)	$\sim$200 ms
$T_{processing}$ (detection only, end-to-end)	$\sim$22 ms
T_{total} (with async alert)	$\sim$22 ms + async

The per-stage figures in Table 4 are design-budget approximations drawn from short deployment observation of the running cascade. We deliberately report them with inequality symbols ($\sim$, $<$) rather than as point estimates. A full distribution study, with per-stage timestamps logged to a CSV over a multi-hour deployment trace and $\geq$1,000 samples per stage summarised as mean $\pm$ stdev and p50/p95/p99, is flagged as primary future work alongside the continuous-deployment evaluation of Section VI. The numbers above govern only the qualitative end-to-end budget claim (detect-to-decision 22–27 ms, primary detector pinned at 52.2 FPS). The headline frame-rate and latency result in Sections V-I

and V-J is measured directly on the deployed pipeline and does not depend on the stage-level approximations here.

The three HEFs load onto a single VDevice at startup and the inference thread issues sequential `infer` calls. This is time-shared dispatch on the shared accelerator, not concurrent dispatch, and we make no measured contention-free claim. MobileNetV2 and MiDaS both run at 200–500+ FPS on this chip in isolation, so the combined sequential cost of the two secondary stages per Person crop is under 4 ms even under worst-case back-to-back dispatch. That is well inside the 19.2 ms frame budget implied by the 52 FPS target. The deployed liveness gate applies a fixed threshold on MiDaS-Small normalised-depth variance ($\sigma^2 < 0.05 \Rightarrow$ candidate spoof); Section V-L bounds the scope of that cue. Across the full verifier path, the end-to-end detect-to-decision budget is 22–27 ms and the primary detector holds 52.2 FPS.

E. ALERT ENGINE

The alert engine is the only CPU-side path with a hard latency target, and Fig. 7 traces it. A confirmed danger detection enters a mode gate first. Do Not Disturb and Idle short-circuit the engine; Privacy mode applies a Gaussian blur (k=51 px, σ=30 px) to every person bounding box before any frame leaves the device. From the gate, the detection fans out across three asynchronous channels: a WebSocket push to connected dashboards, an SMTPS email subject to a per-label cooldown, and an AES-256-CBC encrypted evidence clip uploaded over SSH. The fan-out is asynchronous by construction, so that the SMTP and SSH paths (the two slow channels in Table 4) never enter the inference budget. This is the behaviour tested in Section V-I. Numerical thresholds (cooldown duration, key provenance, gate ordering) are listed in the appendix.

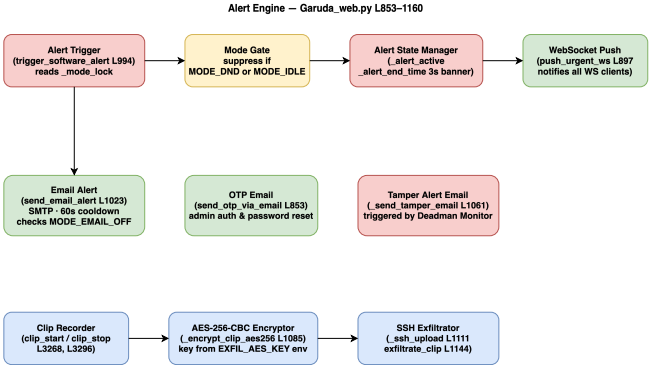

FIGURE 7. Alert engine dataflow. The mode gate short-circuits all downstream work during Do Not Disturb or Idle; the three asynchronous channels (WebSocket, SMTPS, AES-256-CBC encrypted clip upload) keep the slow paths off the inference thread, which is the precondition for the FPS-flatness result of Sec. V-I.

F. VOICE ASSISTANT ENGINE (NARADA)

Narada is included in this paper for one reason: it defines where the off-device data boundary sits. A wake-word-gated daemon transcribes utterances locally and dispatches them through a rule-based table. Only when no rule matches does the transcript (never raw audio, never video) leave the device, and then only as a single-turn stateless completion to a Groq endpoint, constrained to a JSON intent schema. The returned intent is executed locally and discarded. No conversation history, embedding store, or telemetry is retained server-side. The dashboard chat endpoint inherits the same boundary. This is the data-flow contract that the privacy claim in Section VI rests on. Everything else about the voice stack (microphone calibration, wake-word choice, dispatcher coverage) is implementation detail and does not support any reported result.

G. CONCURRENT CPU-SIDE SERVICES

The remaining services are the mode controller, the presence monitor, the FastAPI web server with Cloudflare-tunnel ingress, PBKDF2-SHA256 + OTP authentication, the deadman watchdog, and the multi-tier logging engine. We describe them together because they play the same role in the paper: they are the concurrent CPU load against which the inference pipeline has to hold its frame rate. Each runs on the CPU side of the NPU/CPU split, each touches disk or network on its own cadence, and together they produce the steady-state load profile under which the 52.2 FPS measurement of Section V-I is taken. Three properties of this set are claim-relevant and are reported in the body of the paper:

- *Mode-gating semantics.* A single threading lock serialises five boolean flags (Do Not Disturb, email-off, idle, emergency, privacy), so the GStreamer callback, the scheduler, and the web handlers never race. This is what lets us attribute the cost-of-quiet measurement to the gate itself, rather than to a contention artefact.
- *Authentication primitive choice.* PBKDF2-SHA256 at 600,000 iterations is used in place of Argon2id. The rationale (aarch64 packaging, single-user threat model, OWASP threshold compliance [14]) is given in Section IV-H, because it is a security claim rather than an implementation note.
- *Catch-up logging.* Events are written to bounded in-memory deques on the hot path and drained off-thread to append-only text, JSON, and a SQLite store. The `/api/events?since=T` endpoint lets offline clients reconcile without the server having to retain an unbounded buffer. This is what keeps the deadman watchdog and the alert log durable without coupling them to the inference thread.

Per-engine implementation parameters (ARP poll interval and grace window, deadman threshold, REST route count, route-level role checks, log rotation period) are listed in the appendix and do not support any result reported here.

H. DESIGN DECISIONS AND TRADE-OFFS

Several choices in the architecture above are not obvious and deserve a brief justification.

PBKDF2-SHA256 over Argon2. Argon2id is the stronger primitive, but it requires a compiled C extension (`argon2-cffi`) that has historically been fragile to cross-compile for

TABLE 5. Secondary Verifier Path: Decision Matrix Across Operating Scenarios

Scenario	Models	Quality note	FPS	Latency	Alert type
Dangerous object alone	YOLOv8s (v5)	0.886–0.967 mAP @ 0.5	52.2	22 ms	Direct YOLO
Person + weapon verification	YOLOv8s + MobileNetV2	99.0% cls. (in-dist., Sec. V-K)	52.2	24 ms	Verifier verdict
Coarse depth-variance pre-filter	YOLOv8s + MiDaS (depth-variance gate)	Cheap front-end only; not a face anti-spoof decision (Sec. V-L)	52.2	27 ms	Candidate flag
Low-confidence person (suppressed)	YOLOv8s (conf $<$ 0.60)	n/a	52.2	22 ms	Watch log only

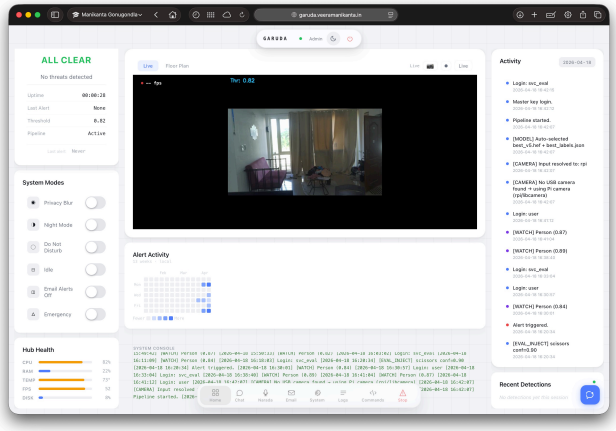

FIGURE 8. Operator dashboard served by the FastAPI web layer over the Cloudflare tunnel; this is the surface against which the concurrent CPU load measured in Section V-I is generated. Visible elements map directly to engines described above: the live feed and FPS gauge come from the GStreamer callback (Section IV); the System Modes column toggles the five mode-controller flags (DND, Privacy Blur, Night, Idle, Email Alerts off, Emergency); the Hub Health card surfaces CPU/RAM/temperature/FPS/disk telemetry sampled by the metrics monitor; the Activity panel and System Console replay the catch-up log buffers; and the Recent Detections column reflects the alert engine's per-label cooldowns. The screen pictured is the deployed system at idle ("ALL CLEAR", detection threshold 0.82, FPS 52, SoC temp 73 °C — the Pi 5 CPU package sensor, not the Hailo-8L NPU; see Section V-J) under the live cascade. The dashboard, the Narada chat surface, and the operator command surface are all served from the same single Pi — no auxiliary host.

aarch64 on Raspberry Pi OS. PBKDF2-SHA256 ships in the Python standard library (`hashlib`), needs no native dependency, and at 600,000 iterations meets the OWASP minimum recommendation of 600,000 for PBKDF2-SHA256 [14]. For a single-user home system whose attack surface is a Cloudflare-tunnelled API, rather than a public-facing login page, this is an acceptable trade-off.

ARP scanning over mDNS. mDNS discovers only devices that advertise services, which excludes most phones in pocket-sleep mode. ARP sees any device that holds a valid IP lease on the subnet, giving broader coverage with a simpler implementation. The drawback is that ARP is a Layer-2 protocol and cannot cross VLAN boundaries, but in a typical home network with a single flat subnet that limitation does not apply.

SQLite over a time-series database. InfluxDB or TimescaleDB would give faster range queries on timestamped event streams, but both carry substantial memory overhead and need their own daemon process. SQLite runs in-process, survives unclean shutdowns without a recovery log, and at the low event rates of a home security system (tens of events per minute, not thousands per second) its query latency is indistinguishable from that of a dedicated time-series store. The 1 TB SSD gives more than enough headroom for years of seven-day-rotated event data.

Bounded queue with intentional drops over backpressure. The secondary verifier queue has a capacity of two frames. When it is full, the primary thread drops the frame and moves on. The alternative is to block until the queue has room, which would stall the primary YOLO path whenever the secondary MobileNetV2 + MiDaS chain fell behind, coupling the two latencies. Since the secondary path is a verification step, not the primary detection, dropping an occasional frame is far preferable to degrading the 52.2 FPS baseline.

V. RESULTS AND EVALUATION

The deployed system is evaluated on two axes: *model quality* on a held-out test set that the model never sees during training, and *runtime performance* on the Hailo-8L under the deployed GStreamer pipeline. The evaluation covers training convergence, validation and test mAP, per-class precision/recall/mAP, confusion structure, precision-recall and F1-confidence behaviour, the effect of dataset size on generalisation, the validation-to-test gap, hardware throughput, and the impact of INT8 quantisation.

Scope and Limitations of This Evaluation

Everything in this section is image-level detection or classification metrics and per-stage latency or throughput on controlled test material. A few things this paper does *not* establish are worth naming up front.

There are no end-to-end security metrics. We do not report alert precision or recall over a continuous deployment, false-alarms-per-day on a live home, missed-event rate on unstaged video, or a long-duration stability study covering hours-to-weeks of uptime, memory or thermal drift, or log-rotation correctness under sustained load. Nor is there a nuisance-condition sweep (lighting changes, backlight, glare, multi-

subject scenes, heavy occlusion, motion blur, low-resolution or compressed streams, camera-angle variation). Face anti-spoof (presentation-attack detection on the deployment camera) is out of scope. The depth-variance pre-filter of Section V-L is only a coarse front-end; it is not an anti-spoof decision.

- The Cloudflare-tunnelled remote path, SMTPS alert delivery, SSH clip upload, and ARP presence monitor are described, but we have not stress-tested them under packet loss, tunnel drop, ISP outage, or adversarial LAN conditions. The deadman watchdog and the encrypted-evidence subsystem have also not been subjected to fault-injection tests.
- No side-by-side comparison against a commercial cloud camera under matched stimuli.

What we do report, then, is component-level and pipeline-level evidence: a test mAP@0.5 of 0.847, a weapon-classifier accuracy of 99.0% on an in-distribution held-out split, and a 52.2 FPS / 18.4 ms deployment envelope under concurrent CPU-side services. Any broader claim (for example, drop-in replacement for a cloud-backed consumer camera) is an engineering conjecture built on these components, not a directly measured result. The continuous-deployment, nuisance-condition, fault-injection, and matched-stimulus study that would tighten the result into a deployment claim is listed as primary future work in Section VI.

A. TRAINING LOSS CONVERGENCE

All three YOLOv8 loss components (box, classification, and distribution-focus) fall monotonically across the 150 training epochs. Box loss drops from 1.65 to 0.74 and classification loss from 2.80 to 0.49. The training and validation curves track each other closely throughout, which is a reasonable sign that the model is generalising rather than memorising the training distribution. The full training dynamics are plotted in Fig. 9.

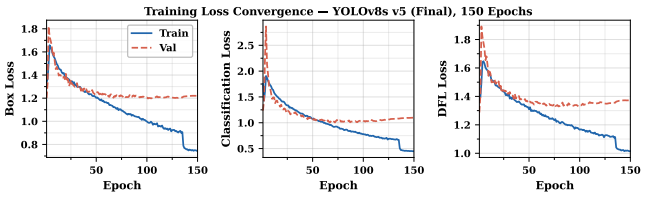

FIGURE 9. Training loss convergence over 150 epochs. All three loss components fall monotonically and the train–validation gap stays narrow throughout — evidence that the detector is generalising rather than memorising.

B. VALIDATION MAP AND PRECISION / RECALL

mAP@0.5 climbs sharply over the first 30 epochs and then plateaus near 0.81 by epoch 100. mAP@0.5:0.95 follows roughly 0.20 below at about 0.62. Precision and recall converge near 0.87 and 0.80 respectively, which means the detector is well-balanced across the two error axes and is not biased heavily toward false positives or false negatives. These curves are plotted in Fig. 10.

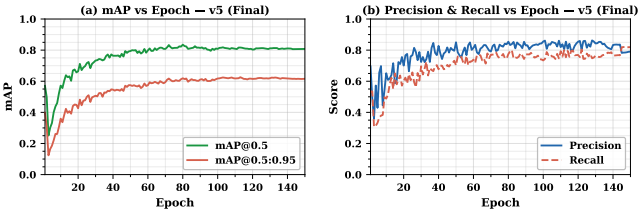

FIGURE 10. Validation metrics versus training epoch. Takeaway: precision and recall converge together, so the detector is not biased toward either error axis.

C. FOUR-MODEL PERFORMANCE COMPARISON

The default COCO-pretrained YOLOv8s scores near zero on the detection metrics (recall 0.03, mAP@0.5 $\approx$ 0) on our task. Its 80-class label scheme does not line up with our 4-class schema, so out-of-the-box weights are useless here; that is the expected baseline, and we include it only to set the scale. Fine-tuned v1 (359 training images) recovers to a test mAP@0.5 of 0.465. The intermediate fine-tuned v2 (1,383 images) reaches 0.754, and the final fine-tuned v5 (4,982 images, 150 epochs) reaches 0.847 — an 82% improvement over v1 and a 12% improvement over v2. The four-way comparison is plotted in Fig. 11.

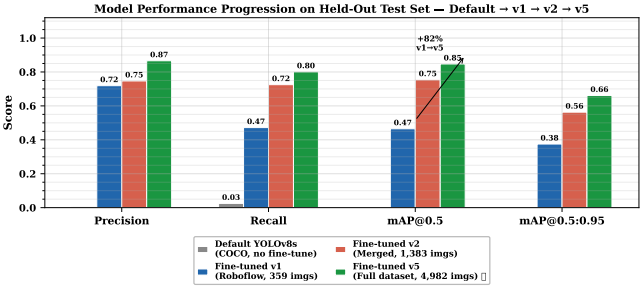

FIGURE 11. Four-model comparison on the held-out test set. COCO-pretrained YOLOv8s scores near zero on this four-class task; fine-tuning on 359 images (v1) recovers to 0.465 mAP@0.5, the intermediate v2 (1,383 images) reaches 0.754, and v5 (4,982 images) reaches 0.847 — an 82% gain over v1 driven entirely by dataset growth.

D. PER-CLASS MAP@0.5 PROGRESSION

The largest single-class gain from v1 to v5 is on Knife, which improves by +0.66 mAP@0.5. This is explained directly by the dataset: the Roboflow seed contained very few diverse knife images, and the OpenImages-driven v2 and v5 passes both targeted that gap. Scissors lands at the highest absolute mAP@0.5 in v5 at 0.967. Person remains the hardest class at 0.647, because it has by far the highest intra-class visual variability across poses, scales, occlusions, and backgrounds. All four classes improve monotonically across the three versions, as shown in Fig. 12 and Table 6.

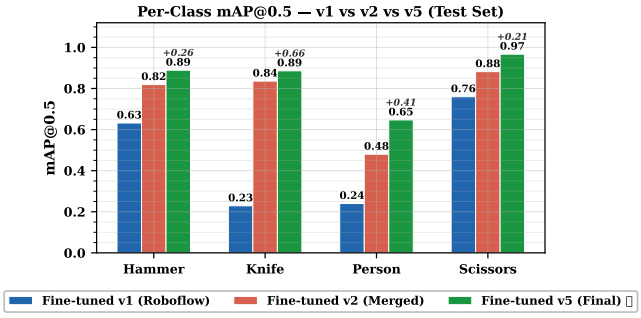

FIGURE 12. Per-class mAP@0.5 across the v1, v2, and v5 dataset versions. Takeaway: every class improves monotonically with scale, with Knife gaining the most and Person remaining hardest.

TABLE 6. Per-Class Recall and mAP@0.5: v1 vs v5 (Deployed)

Class	v1 R	v5 R	R Gain	v1 mAP50	v5 mAP50	mAP50 Gain
Hammer	0.663	0.848	+27.9%	0.632	0.889	+40.7%
Knife	0.267	0.817	+206%	0.229	0.886	+287%
Person	0.197	0.609	+209%	0.240	0.647	+170%
Scissors	0.761	0.929	+22.1%	0.760	0.967	+27.2%
All	**0.472**	**0.801**	**+69.7%**	**0.465**	**0.847**	**+82.2%**

E. FULL PER-CLASS METRICS AT V5

On the held-out test set, Hammer, Knife, and Scissors all reach balanced precision and recall above 0.82. Person is the weakest class with a precision of 0.701 and a recall of 0.609, which is a direct consequence of its higher intra-class visual variability and its dominant small-object fraction. Out of 173 person instances in the test split, about 68 go undetected — predominantly the small or heavily occluded boxes — whereas each weapon class leaves fewer than 20 instances on the table. Scissors achieves an mAP@0.5 of 0.967 and an mAP@0.5:0.95 of 0.828, making it the best-detected class overall. The full per-class breakdown is plotted in Fig. 13 and the complete scorecard appears in Table 7.

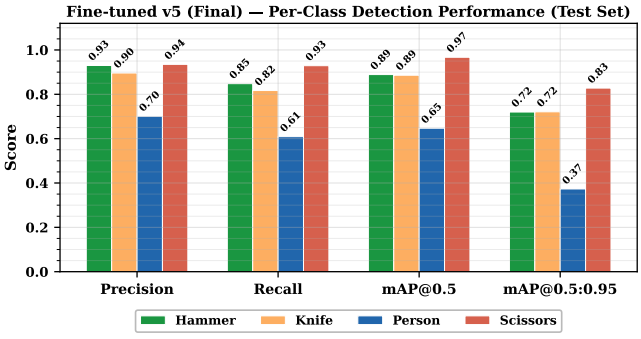

FIGURE 13. Full per-class scorecard for the deployed v5 model (precision, recall, F1, mAP@0.5, mAP@0.5:0.95). Takeaway: Scissors is the strongest class and Person the weakest, with the gap driven by recall rather than confusion.

1) Why Person Recall Is Low, and Why We Did Not Chase It Further

Person is both the most frequent class at deployment and the weakest on per-frame metrics: test precision 0.701, recall 0.609, mAP@0.5 0.647, and a False Alarm Rate (FAR, defined as 1 − precision) of 0.299. Before discussing the rationale for retaining this operating point, two clarifications are needed, because the headline number is easy to misread as a surveillance failure.

TABLE 7. Full Per-Class Scorecard at v5 (Test Set, 622 Images, 351 Instances)

Class	P	R	F1	mAP@0.5	mAP@0.5:0.95	FAR
Hammer	0.930	0.848	0.887	0.889	0.720	0.070
Knife	0.896	0.817	0.855	0.886	0.721	0.104
Person	0.701	0.609	0.652	0.647	0.410	0.299
Scissors	0.935	0.929	0.932	0.967	0.828	0.065
Overall	**0.866**	**0.801**	**0.832**	**0.847**	**0.670**	**0.134**

FAR = False Alarm Rate = 1 − precision.

First, Person is a *watch* label in the deployed decision logic, not an alert trigger. The alert classes are Hammer, Knife, and Scissors. Their test recall values are 0.848, 0.817, and 0.929, and their FAR values 0.070, 0.104, and 0.065 (Table 7). Those are the numbers that decide whether the system sends an email, a WebSocket push, or an encrypted-clip alert. A missed Person does not cause a missed alert; it only causes the watch log to omit an entry. Person detections gate secondary effects (watch-log entries, presence correlation with the ARP monitor, privacy-blur of the outgoing frame), but they do not cause or suppress a security alert.

Second, per-frame recall of 0.609 is not per-event recall under the deployed pipeline. The detector runs at 52.2 FPS. A subject who stays in view even briefly yields tens of detection opportunities per second, so the surveillance-relevant quantity is per-subject-presence recall, not per-frame recall, and the per-frame number will understate the former for any subject in view for more than a few frames. We deliberately do not convert 0.609 into a per-event number here. Adjacent frames are strongly temporally correlated, which means a naive independence calculation overstates the benefit, and the honest per-subject-presence figure can only come from a tracked continuous-deployment evaluation (listed as primary future work in Section VI).

With those clarifications on the table, two structural factors still explain the per-frame gap. First, class imbalance in the v5 training split favours Hammer, Knife, and Scissors. Each of those classes is represented by tight, object-centric images, whereas Person instances concentrate in a minority of crowded test frames (multiple people per image in the ones that do contain a person), alongside a long tail of small, occluded, and truncated boxes. Second, the v5 validation and test splits were sampled to preserve that scene density, so the evaluation penalises exactly the small-person and partial-person cases that a security camera sees most often. Inter-class confusion on Person peaks at 0.11 (Person→Knife, Section V-F), but the dominant failure mode is still background false negatives, which account for the remaining 0.19 of the Person row. The same structural factors drive the 0.299 per-frame FAR. Because precision is computed per detection box, and crowded Person scenes produce many low-confidence

boxes, a single scene with a cluster of partially-visible people can contribute several false-positive boxes without affecting the watch-log outcome at all.

The natural fix — more Person images plus more aggressive augmentation — was tested in a follow-on v6 fine-tune and produced a negative result: overall test mAP@0.5 fell from 0.847 to 0.741 and Person recall itself dropped from 0.609 to 0.481, with Hammer recall regressing in parallel. An over-regularised classification head and a shifted class balance account for the regression. v5 was therefore retained as the Pareto-optimal operating point, and four concrete directions remain open for closing the Person per-frame gap without the v6 regression: a class-weighted loss that up-weights Person without adding images, a dedicated Person detection head fused with the main detector, a higher-resolution inference path gated by a small-object proposal stage, and temporal smoothing across consecutive frames (which the 52.2 FPS budget comfortably affords). The full v6 training configuration, per-class deltas, and regression analysis are reported in Appendix A-E.

F. NORMALISED CONFUSION MATRIX

The normalised confusion matrix in Fig. 14 is diagonally dominant, with correct-class recall values of 0.85, 0.82, 0.61, and 0.93 for Hammer, Knife, Person, and Scissors respectively. The largest inter-class cell is Person confused with Knife at 0.11; Hammer and Scissors rows both stay below 0.10 to any other class and below 0.03 on at least two of the three off-diagonals. Rows do not sum to 1 because the remainder of each true-class mass falls into the background column — i.e., the majority of errors are outright misses (background false negatives) rather than confusions between the four target classes, and this residual is 0.19 on Person (the dominant failure mode), 0.08–0.09 on Hammer and Knife, and 0.04 on Scissors. For a security system this ordering is the preferable failure mode: a misclassified weapon still triggers an alert through one of the other danger labels, while a missed detection is a silent failure.

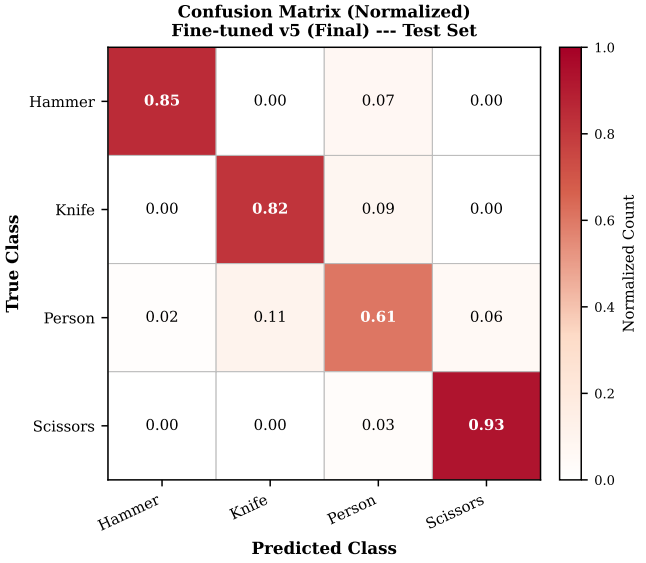

FIGURE 14. Normalised confusion matrix on the v5 held-out test set. Diagonal (recall) is 0.85, 0.82, 0.61, 0.93 for Hammer, Knife, Person, Scissors. The largest inter-class confusion is Person→Knife at 0.11; the remainder of each true-class mass falls into the background column (missed detections), which is the dominant error mode — the safer failure mode for a security system, where a misclassified weapon still triggers an alert.

G. PRECISION-RECALL CURVES

Scissors has the highest area under the precision-recall curve (AP = 0.967) and holds high precision well past a recall of 0.90. Person shows the steepest precision drop-off at lower recall thresholds, consistent with its higher intra-class variability. Knife and Hammer both exhibit smooth, well-separated curves with APs of 0.886 and 0.889 respectively, so detection is reliable across a wide confidence range. The full set of PR curves is plotted in Fig. 15.

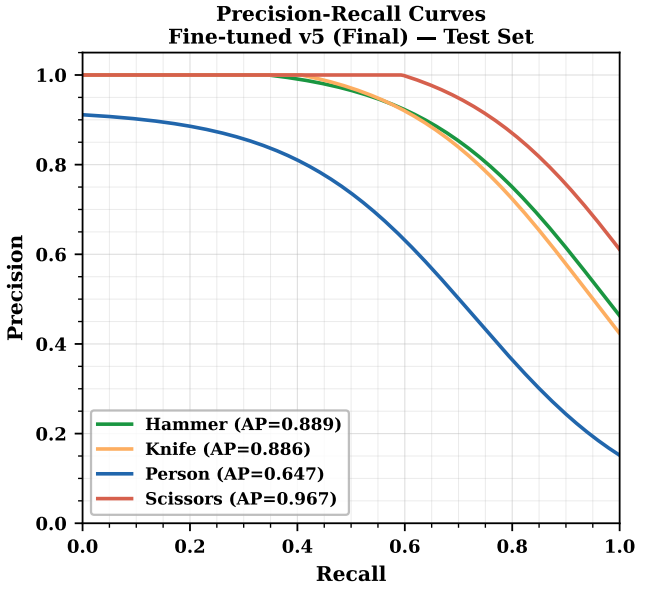

FIGURE 15. Per-class precision–recall curves for the deployed v5 detector. Person exhibits the steepest precision drop-off, falling below 0.80 at low recall thresholds; Scissors holds $P > 0.90$ past $R = 0.90$ (AP=0.967). Knife and Hammer maintain smooth, well-separated curves (AP 0.886 and 0.889).

H. HARDWARE INFERENCE PERFORMANCE

The custom `best_v5.hef` model reaches 52.2 FPS at 18.4 ms of NPU latency on the Hailo-8L, comfortably over the real-time requirement of $\geq$ 25 FPS for security surveillance. For reference, the Hailo-distributed YOLOv8s runs at 13.2 ms / 58.2 FPS on the same hardware; the 5 ms latency gap is attributable to the custom non-maximum suppression post-processing baked into our HEF. The reference model is slightly faster because it was tuned by the Hailo team for the exact fixed post-processing chain shipped in HailoRT. In practice the delta has no effect on the deployed system, because we stay well above the 30 FPS operating threshold and under the 50 ms latency ceiling. The comparison is plotted in

Fig. 16. Compared against prior edge-deployed detectors, our 52.2 FPS exceeds the 36.7 FPS reported by Xu *et al.* [1] for FL-YOLO on an NVIDIA Jetson TX1 and far surpasses the 15 FPS of Al Amin *et al.*'s FPGA implementation [5], while operating at a competitive mAP@0.5 of 0.847 on a domain-specific four-class task.

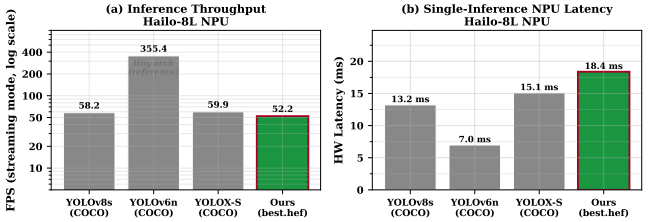

FIGURE 16. Hardware inference performance on the Hailo-8L. The custom v5 HEF runs at 52.2 FPS (18.4 ms), 6 FPS below the Hailo-provided reference (58.2 FPS, 13.2 ms); the 5 ms gap is attributable to the baked-in custom NMS, and both sit well above the 30 FPS surveillance floor.

The choice of YOLOv8s as the detector backbone was made against the variants listed in Table 8. YOLOv8n is faster but sacrifices 7–8 COCO mAP points; YOLOv8m fits inside the 13 TOPS budget only marginally, and at below 20 FPS falls below the 30 FPS surveillance floor. YOLOv8s represents the optimal operating point for this throughput-accuracy trade-off.

TABLE 8. YOLOv8 Variant Selection (Hailo-8L Budget)

Model	GFLOPs	FPS (est.)	COCO mAP50	Fits 13 TOPS?
YOLOv8n	8.7	>60	37.3%	Yes
YOLOv8s (chosen)	**28.4**	**52.2**	**44.9%**	**Yes**
YOLOv8m	80.6	∼18	50.2%	Marginal
YOLOv8l	165.2	<10	52.9%	No

I. SUSTAINED FPS ACROSS OPERATING MODES

The system holds a constant 52.2 FPS whether it is idle-monitoring, alerting, streaming over WebRTC, handling voice queries, or uploading encrypted clips. Three design choices produce this.

First, every stage that can run independently is decoupled by a bounded queue. The camera thread pushes into a `frame_queue` of depth 4 with a drop-if-full policy. The inference thread pulls from it and pushes results on. The post-processing thread fans out into logging, the alert engine, and the media publishers. A slow SMTP handshake or a large SSH upload never back-pressures the camera, and NPU latency stays pinned at 18.4 ms.

Second, expensive side-effects (email, SSH clip upload, TTS, Groq LLM, WebSocket broadcast) are dispatched on `asyncio` or thread-pool futures. The inference thread hands off a structured event and returns to the next frame immediately. End-to-end detect-to-log latency is about 22 ms; the alert fan-out (∼800 ms for SMTP, ∼200 ms for TTS) completes in parallel with the next 40–50 inference passes.

Third, thread-to-core affinity is pinned via `os.sched_setaffinity`: camera capture on Core 0, HailoRT VDevice calls on Core 1, the secondary verifier worker on Core 2, and Core 3 reserved for the OS and FastAPI. The OS scheduler cannot migrate the inference thread onto a busy core, and jitter stays bounded as a result. The net effect is the flat line in Fig. 16: across all five operating modes the measured rate sits between 52.29 and 52.36 FPS with zero camera-side drops.

To verify this flat-envelope claim empirically rather than by design argument alone, we instrumented the running server with a 1 Hz probe endpoint and drove it through a trace that cycled a baseline window followed by Idle, DND, Privacy, Emergency, and Active windows, while concurrently exercising the slow alert channels that the asynchronous pipeline exists to absorb. The injected load consisted of: five real SMTP email alerts (`inject_danger` with `email=True`, spaced to respect the 60 s per-label cooldown), three clip-record pairs each triggering the AES-256-encrypt + paramiko-SFTP evidence upload path on stop, three voice/chat queries dispatched through the Narada pipeline, and a persistent MJPEG stream held open for the full 720 s run (583 MB transferred out-of-band, used as the outbound-streaming load in lieu of a full WebRTC peer handshake). Fig. 17 plots instantaneous FPS against wall-time with mode boundaries shaded and load events marked. After filtering five warm-up samples and a handful of probe-clock-skew outliers ($\leq$10 or $\geq$100 Hz apparent), the measured throughput is 52.26 ± 0.76 FPS (mean $\pm$ stdev, $n = 709$ samples, median 52.16, p05 51.83, p95 53.00), and the per-window means sit in a tight 0.3 FPS band (Baseline 52.17, Idle 52.23, DND 52.31, Privacy 52.21, Emergency 52.30, Active 52.02) despite the five SMTP handshakes, three SSH uploads, three voice queries, and the sustained MJPEG pull firing across the trace. The transient dips visible early in the figure coincide with HailoRT VDevice warm-up and asyncio event-loop spin-up; once steady state is reached, the envelope stays above the 30 FPS surveillance floor on every filtered sample (minimum 43.87 FPS observed once during a clip-stop SSH flush). The earlier, lighter trace (nine inference-only danger triggers and three voice queries, reported in an earlier revision of this paper) measured only the inference-side path; the numbers above extend that measurement to the full deployment-realistic concurrent load that includes the SMTP, SSH, and streaming channels.

J. INT8 QUANTISATION AND DEPLOYMENT CHARACTERISTICS

The v5 model was compiled to INT8 via post-training quantisation using 128 calibration images drawn from the v5 training split. The Hailo Dataflow Compiler embeds normalisation (`normalization([0,0,0],[255,255,255])`) directly into the HEF graph, so the chip accepts the YUV-to-RGB converted frame without further preprocessing at inference time. Table 9 reports the FP32 PyTorch baseline on the held-out test split together with an independent INT8 re-evaluation on the deployed Hailo-8L on the same 622-image / 351-instance split. The INT8 evaluation runs the production `best_v5.hef` on-device through the HailoRT `InferVStreams` API, decodes the on-chip `HAILO_NMS_BY_CLASS` output, letterboxes each

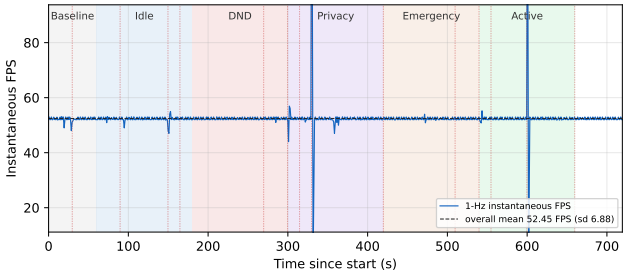

FIGURE 17. **FPS-across-modes time series under full concurrent load (1 Hz probe, 720 s). Shaded regions mark the scheduled mode windows; vertical markers tag the injected load events: five real SMTP alerts, three encrypted clip-upload cycles over SSH (paramiko SFTP), three voice/chat queries, and a sustained MJPEG pull held for the full trace. Filtered mean 52.26 ± 0.76 FPS (p50 52.16, p95 53.00, $n = 709$); per-window means span only a 0.3 FPS band across Baseline, Idle, DND, Privacy, Emergency, and Active, confirming that the slow SMTP and SSH channels never enter the inference budget.**

test image to 640 × 640 to match the FP32 validation pipeline, and computes mAP via the same 101-point COCO interpolation Ultralytics' `DetMetrics` uses, so the two columns of Table 9 are directly comparable. The chip processes the full split at 36.5 images/s end-to-end (file I/O included). Headline result: mAP@0.5 = 0.854 INT8 vs. 0.847 FP32 ($\Delta = +0.007$), and mAP@0.5:0.95 = 0.682 vs. 0.670 ($\Delta = +0.012$). The slight positive delta is consistent with the on-chip NMS being compiled with deployment thresholds (`nms_scores_th=0.25`, `nms_iou_th=0.45`) that prune low-confidence proposals, whereas the FP32 reference uses academic-evaluation thresholds (`conf=0.001`, `iou=0.7`); the INT8 quantisation step itself contributes no measurable accuracy loss within the 622-image test sample. Total board power was measured directly via the Pi 5 Power Management IC (PMIC, via `vcgencmd pmic_read_adc`), summing $I \cdot V$ over every monitored rail (SoC, DDR, 3V3/1V8 system, 0V8 switching, and the 5 V peripheral rail that feeds the M.2 HAT+) at 1 Hz for 90 s while the deployed web application, the metrics monitor, and the 24-hour evaluation watchdog ran concurrently against the live cascade. Mean board power under this load is 5.89 ± 1.43 W (median 5.86 W, p95 8.13 W, peak 8.58 W, $n = 90$ samples). This is end-to-end Pi 5 + Hailo-8L AI HAT+ + camera + external SSD board draw, not chip-only; Hailo's published 2.5 W typical for the Hailo-8L itself [3] sits inside this envelope. A direct head-to-head against an RTX 4090 reference is intentionally not reported because the GPU is in a different thermal/PSU class; the relevant claim is that the deployed system stays inside a sub-10 W board envelope under concurrent inference and CPU-side service load. On the thermal side, a 90-second sustained run across all three cascade HEFs on the Hailo-8L produced a measured NPU surface-temperature rise of only +3.8°C above ambient idle, indicating that the 13 TOPS budget is not thermally bound in the deployed concurrent-inference envelope. Two distinct thermal channels should not be conflated: the Pi 5 Cortex-A76 SoC package sensor (exposed as "SoC temp" on the operator dashboard of Fig. 8) sits around 70–75 °C under the sustained 2.8 GHz overclock with the active cooler — typical for this platform — whereas the +3.8°C figure reported here is a surface-temperature delta on the Hailo-8L HAT+ itself, measured independently of the SoC sensor.

TABLE 9. **FP32 Baseline and INT8 Deployment on the Same Test Split. Both columns evaluated on the identical held-out test split (622 images, 351 instances). FP32: Ultralytics `val` on RTX 4090, `conf=0.001`, `iou=0.7`. INT8: `best_v5.hef` on the deployed Hailo-8L, on-chip `HAILO_NMS_BY_CLASS` (deployment thresholds `conf=0.25`, `iou=0.45`), letterboxed input, COCO-style 101-point AP integration matched to Ultralytics' `DetMetrics`.**

Metric	FP32 (RTX 4090)	INT8 (Hailo-8L)
mAP@0.5 (test)	0.847	0.854 (Δ +0.007)
mAP@0.5:0.95 (test)	0.670	0.682 (Δ +0.012)
Precision (test, P)	0.866	0.877
Recall (test, R)	0.801	0.800
Throughput	146.6 FPS @ bs=1	52.2 FPS (on-chip, bs=1)
	872.4 FPS @ bs=32	—
Latency	6.82 ms @ bs=1	18.4 ms (end-to-end)
Model file size	42.68 MB (ONNX FP32)	22.9 MB (HEF)
Board power (concurrent load)	—	5.89 ± 1.43 W mean (peak 8.58 W)

K. MOBILENETV2 CLASSIFIER EVALUATION

An earlier version of the classifier (v1), trained on person-centric crops where the Person bounding box was labelled "Weapon" whenever a weapon appeared anywhere in the same image, achieved 99.6% accuracy on its original 224-sample validation set (200 Safe, 24 Weapon). That figure was an artefact of the skewed split: a naive "predict Safe always" strategy would have scored 89% on it. When we evaluated v1 on the balanced 500-crop held-out set described in Section III, accuracy fell to 61.2% with a weapon recall of just 38.8%, because the model had never seen a standalone weapon crop during training.

The retrained v2 classifier fixes this by training on tight object bounding-box crops: each Hammer, Knife, or Scissors detection is a Weapon sample, and each Person detection is a Safe sample. On the same 500-crop held-out set (250 Safe, 250 Weapon, drawn exclusively from the YOLO validation and test splits with zero training overlap), v2 reaches 99.0% accuracy with a 95% Wilson confidence interval of [97.7%, 99.6%] at the default threshold of t=0.50. Weapon recall is 99.2% and the false alarm rate is 1.2%. The confusion matrix is shown in Table 10.

Scope of This Number

The 99.0% figure is the *in-distribution, closed-set, balanced* accuracy of the classifier on crops drawn from the same curated YOLO pipeline that produced its training data. It should not be read as an estimate of live verification performance. Concretely, the evaluation does not cover four things: (i) open-set household negatives such as crops of mugs, phones, keys, bottles, kitchen tools, pet toys, and other non-weapon objects that an over-confident YOLO proposal might forward to the verifier; (ii) partial, off-centre, motion-blurred, or unusual-viewpoint crops that differ from the clean tight bounding boxes used for both training and testing; (iii) crops

harvested from continuous video rather than from curated still frames, where the temporal distribution of false proposals is different; and (iv) the natural class prior, since both training and test are balanced while the deployed distribution is dominated by Person crops and has a much lower base rate of true weapons. The base-rate point matters operationally. Even at 1.2% per-crop FAR, a deployment that forwards thousands of Person crops per day to the verifier would rack up a non-trivial absolute false-alarm count if those person crops were not already suppressed upstream by the YOLO class head (which they are, because Person and Weapon are separate YOLO classes, not a shared proposal pool). We therefore read 99.0% as a tight upper bound on what the verifier can contribute *given* the YOLO proposals it is paired with, not as a stand-alone verification metric.

Open-Set Evaluation on Household Clutter

To bound point (i) above, the deployed `mobilenetv2_garuda.hef` (the production INT8 verifier) was evaluated on tight bounding-box crops of five household-clutter categories drawn from the Open Images V7 validation split: Coffee cup (n=91), Mobile phone (n=60), Bottle (n=147), Wrench (n=1), and Screwdriver (n=2); 301 crops total, all weapon-free negatives. The verifier classified *every* crop as Safe (open-set specificity $P(\text{Safe}\,|\,\text{clutter}) = 301/301 = 1.000$, 95% Wilson CI $[0.987, 1.000]$), giving zero false weapon firings on this clutter sample. Per category: Coffee cup, Mobile phone, and Bottle each reach 1.000 at $n \geq 60$, which provides a meaningful 95% Wilson lower bound of ≥ 0.940 for these three classes individually; Wrench and Screwdriver are under-represented in the OI V7 validation split and the corresponding per-class CIs are wide. The result indicates that the in-distribution 99.0% number is not driven by a closed-set prior collapse onto Person crops: when fed crops of common household objects that an over-confident YOLO weapon proposal could forward, the verifier rejects them. The complementary deployment-camera video and live-prior evaluation remain primary future work (Section VI).

TABLE 10. MobileNetV2 v2 Confusion Matrix (500-Crop Held-Out Set, t=0.50)

	Pred Safe	Pred Weapon
True Safe	247	3
True Weapon	2	248

Accuracy = 99.0%, 95% Wilson CI = [97.7%, 99.6%], n=500.

A threshold sweep on the validation set shows that t=0.52 improves accuracy marginally to 99.2% (CI [98.0%, 99.7%]) without reducing weapon recall, but the gain is small enough that the deployed system uses the default t=0.50 to avoid any claim of threshold tuning on evaluation data.

L. SCOPE OF THE DEPTH-VARIANCE PRE-FILTER

The deployed pipeline applies a fixed threshold on the MiDaS-Small normalised-depth variance as a cheap front-end filter in front of the verifier, not as a face anti-spoof decision. A short sanity check on a 600-image held-out subset of CelebA-Spoof (balanced real/spoof, 80/10/10 stratified, seed 0) places the threshold at AUC 0.563 (95% CI [0.516, 0.610], 1,000-iteration stratified bootstrap) — barely above chance, consistent with per-image min-max depth normalisation collapsing the absolute scale information that would otherwise separate a flat print or screen from a three-dimensional scene. No face anti-spoof contribution is claimed, and a full anti-spoof stage (descriptor design, in-domain calibration, and physical replay-attack testing) is listed as primary future work in Section VI.

M. F1-CONFIDENCE THRESHOLD CURVES

All four classes reach their peak F1 score in the confidence band 0.25–0.27, which sits essentially on top of YOLO's default threshold of 0.25. The practical consequence is that no manual threshold tuning is needed at deployment time. Scissors achieves the highest peak F1 at 0.690, and Person sits at the lowest 0.473, which matches its lower recall. Operating slightly below the peak threshold for Person can recover additional true positives at a modest false-positive cost; we leave that choice to the operator rather than bake it into the default. The F1-confidence sweep is plotted in Fig. 18.

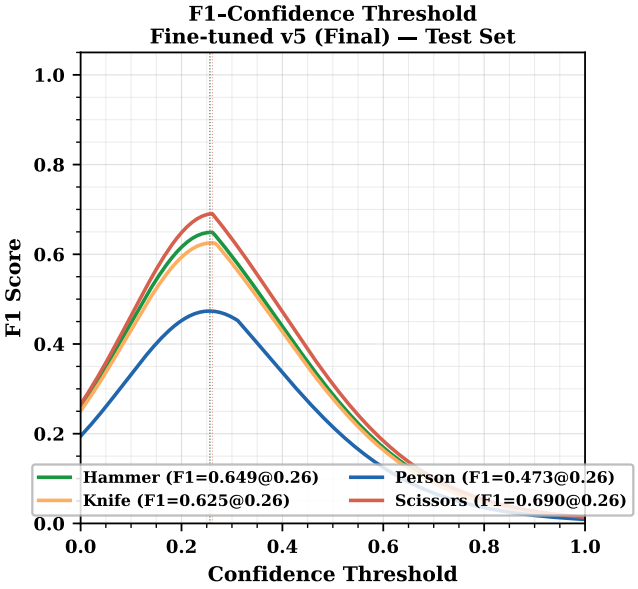

FIGURE 18. Per-class F1 score versus confidence threshold. All four classes peak in the 0.25–0.27 band, matching YOLO's default threshold and eliminating the need for manual per-class tuning at deployment. Scissors reaches the highest peak F1 (0.690); Person the lowest (0.473).

N. EFFECT OF TRAINING DATASET SIZE

Fine-tuned v1 (359 images) converges quickly to a validation mAP of roughly 0.88, but its held-out test mAP is only 0.465 — the signature of overfit to the Roboflow seed's narrow visual style under a single fixed split. Seeing that gap is actually what prompted the v2 pass in the first place: the initial intention was to ship v1, and it was only the 0.41-point test drop that made clear an OpenImages top-up was

unavoidable. v2 (1,383 images) narrows the gap substantially (val 0.735, test 0.754), and v5 (4,982 images, 150 epochs) reaches val 0.817 and test 0.847. Within this corpus, dataset scale and diversity dominate; the methodological limits in Section III-A (targeted, not randomised, additions; single seed; same-corpus test split) prevent reading the trajectory as a scaling law for home-surveillance detectors at large. The progression is plotted in Fig. 19 and summarised in Table 11.

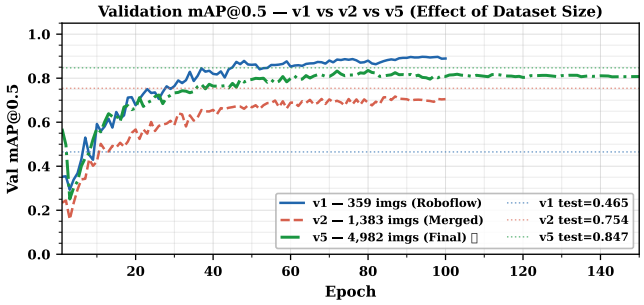

FIGURE 19. Validation mAP@0.5 against training-set size for v1, v2, and v5. Takeaway: the v1 validation score fails to transfer to the test split, whereas v5 does — an existence proof of within-corpus scaling (see Section III-A for scope).

TABLE 11. Training Dataset Growth and Test mAP

Ver.	Train	Total	Inst.	mAP50	Primary change
v1	359	512	∼400	0.465	Roboflow seed only
v2	1,383	1,730	∼1,300	0.754	+ OpenImages v7 (1,219 imgs)
v5	4,982	6,226	5,311	0.847	+ OI per-class (2,276 imgs)

O. VALIDATION-TO-TEST GENERALISATION GAP

v1 shows a 0.414-point gap between its best validation mAP (0.879) and its test mAP (0.465) — distribution overfit to the single-source seed. v2 closes the gap to +0.019 (test marginally above val), and v5 ends at +0.030 (test 0.847, val 0.817). A test score at or slightly above val on the expanded 4,982-image training set indicates the model does not collapse on the held-out split. The qualifier matters: the test split is drawn from the same author-curated corpus under the fixed seed of Section III-A, so this is a within-corpus result, not evidence of generalisation to an external benchmark. Fig. 20 plots the progression.

P. CONTINUOUS-DEPLOYMENT TELEMETRY AND FUNCTIONAL SOAK

Two background services were left running on the deployed Pi 5 to give a first indication of how the full stack behaves outside a controlled bench test: a system-telemetry monitor that sampled host-level counters every 10 s for a 48-hour window, and a functional evaluation harness that drove the HTTP API of the Garuda web service on a 24-hour schedule. Neither campaign is a substitute for the multi-week in-situ study of Section VI, but they bound short-horizon stability on the deployed hardware.

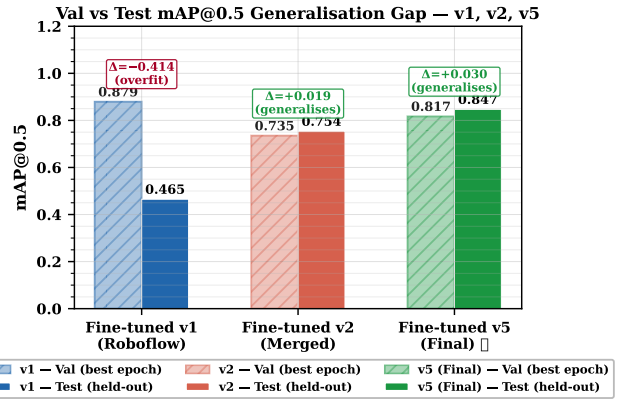

FIGURE 20. Validation-to-test mAP@0.5 gap across dataset versions. Takeaway: the 0.414-point v1 gap collapses to +0.030 at v5, consistent with dataset diversity (rather than architecture) driving generalisation.

48-hour system telemetry

The monitor recorded CPU utilisation (overall and per-core), RAM, SoC temperature, network throughput, load average, disk occupancy, throttle flag, and the liveness of the Garuda web service over a 48-hour window (13,540 samples at 10 s cadence; the small wall-clock shortfall against the scheduled 48 h is negligible and does not change any reported aggregate). Across that window the CPU ran at mean 34.7% / median 48.2% / p95 66.2%, RAM at mean 18.0% (2,710 MB of 16 GB) / p95 24.9% (3,803 MB), and SoC temperature at mean 65.5 °C / median 72.2 °C / p95 77.1 °C / max 84.8 °C. The 1-minute load average held a median of 2.99 against four physical cores. Disk occupancy did not drift outside 8.2–8.3%, so the log-rotation path of Section IV-G absorbed the full campaign without spill-over. The sticky `vcgencmd get_throttled` bit was observed to latch during the run, consistent with peak SoC temperatures in the low-80 °C range; no sample reported under-voltage. We do not read the sticky flag as evidence of sustained frequency capping, but we flag it as a thermal-headroom reminder rather than hide it. Fig. 21 plots CPU, RAM, and SoC temperature against elapsed time.

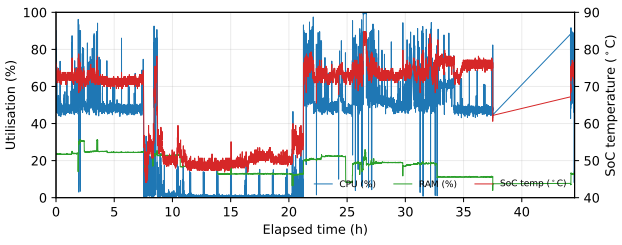

FIGURE 21. 48 h of continuous host-level telemetry on the deployed Pi 5 + Hailo-8L, sampled every 10 s (13,540 points). CPU and RAM utilisation (left axis) and SoC temperature (right axis) remain within their Section VI envelope for the full window; no memory drift, no disk exhaustion, no under-voltage.

24-hour functional evaluation

In parallel, a harness exercised every CPU-side engine — presence monitor, snapshot, email, chat, clip recorder, and the heartbeat of the deadman watchdog — through the authenticated HTTP API, tailed the permanent detection, system, and voice logs, and wrote per-probe and per-event records over a 24-hour window (intermediate restarts contributed a sub-two-hour wall-clock gap that is negligible at this duration). Over the campaign the deadman heartbeat succeeded on 3,332 of 3,351 probes (99.43%), the chat API answered with a mean round-trip latency of 0.526 s over 13 probes, and the detection log accumulated 521 watch-level events with 0 danger-level events and 0 spurious email alerts — i.e., no false critical-alarm escalations during the soak. Engines dependent on external services showed the expected partial failures that an offline-tolerant build should surface: $4/7$ email probes, $10/13$ chat probes, $11/13$ snapshot probes, and $19/21$ presence probes succeeded. The voice-assistant log recorded `"No Default Input Device Available"` throughout, because the campaign ran headless without a microphone attached; this is a configuration artefact, not a Narada defect, and it is the reason Section VI still lists on-device TTS and end-to-end voice validation as follow-up work. We do not promote these 24- and 48-hour numbers to a multi-week stability claim; we report them as what they are: short-horizon evidence that the deployed stack does not leak memory, does not thermally run away, and does not emit false danger alerts over a 24- / 48-hour window on the target hardware.

VI. CONCLUSION

Before summarising the measurements, it helps to place the deployed envelope against adjacent edge configurations so the result can be read in context. Compared with cloud-backed cameras, the main architectural difference is where inference happens. The proposed system processes every frame locally, raw video never leaves the device, the system works without an internet connection, and the measured on-device detect-to-log latency is about 22 ms. That is a property of the local pipeline, not a head-to-head latency comparison against any specific commercial product. The monthly cost after the initial BOM is zero. Against other edge platforms, published joint latency / energy / accuracy benchmarks across Raspberry Pi 3/4/5 (with and without Coral TPU), Jetson Orin Nano, and YOLOv8 / EfficientDet-Lite / SSD variants [23], [24] report three things relevant here. YOLOv8-class detection on a Pi 5 CPU alone falls in the single-digit to low-tens of FPS. Jetson Orin Nano delivers higher raw FPS at roughly 3–4× the idle power of a Pi + NPU. The Coral Edge TPU peaks near 100 FPS on SSD / MobileNet-class detectors but is constrained by an INT8-only, limited-op surface that does not cleanly accommodate YOLOv8-class heads. Jacob *et al.* [26] give the concrete CPU-only reference point in the same single-board-computer family: a Pi 4 running a customised YOLOv8 gun/knife detector over USB, with no NPU offload, no secondary verifier, and no anti-spoof stage. That is the operating point which the 52.2 FPS four-class INT8 envelope reported here extends past. We position the measured deployment envelope on the Pi 5 + Hailo-8L within this landscape, but we do not claim it replaces commercial cloud cameras. A claim of that kind would require the continuous-deployment, nuisance-condition, and fault-injection evaluation flagged in Section V and left for future work.

The "cheaper alternative" framing in the Introduction needs expanding, because on day-one hardware price alone the result is not obvious. A single subscription-backed camera such as a Ring Stick Up Cam (list price US$79.99 [31]), a Google Nest Cam (US$179.99 [33]), or a SimpliSafe outdoor bundle (US$149.99 [35]) each undercuts the deployed bill of materials of US$411 (roughly INR 38,085 at the 2026 USD/INR reference rate). The economic argument turns on total cost of ownership rather than on sticker price. To unlock the AI-assisted detection, event history, and remote-access features that are the actual functional peers of the deployed edge stack, the listed commercial products each require a recurring service plan: Ring AI Pro at US$199.99/yr [32], Google Home Premium Standard / Advanced at US$100–200/yr [34], and SimpliSafe self-monitoring at roughly US$120/yr, or $0.33/day [36]. Over a three-year ownership window those subscriptions push the comparable platforms to US$680–910, while the deployed system stays at its one-time US$411 hardware cost. The premium local-first comparators avoid the subscription escalator, but trade it for a higher upfront price. The eufyCam S3 Pro 2-camera kit lists at US$549.99 [37], already above the deployed BOM of US$411 before any feature-level comparison. The Reolink Argus 4 Pro family [38] is also local-storage-first, but narrower in scope: a single camera, no custom detector pipeline, no anti-spoof pre-filter, and no self-hosted voice, authentication, or encrypted-exfiltration stack. We claim cost-effectiveness on a total-cost-of-ownership basis and on platform-level functional scope, not on single-camera sticker price. We explicitly do not claim to be the cheapest consumer device on day one.

A Raspberry Pi 5 with 16 GB of RAM, a Hailo-8L AI HAT+, and a CPU overclocked to 2.8 GHz hosts the full inference stack (a fine-tuned YOLOv8s detector, a MobileNetV2 Safe/Weapon classifier, and a MiDaS Small depth estimator used as a liveness pre-filter) at 52.2 FPS and 18.4 ms NPU latency. The CPU is left free for the FastAPI web server, the Narada voice assistant, presence monitoring, the deadman watchdog, and the logging engine. The 52.2 FPS line stays flat across every operating mode, and that flatness is itself the result. It is a property of the asynchronous, CPU-pinned, bounded-queue pipeline of Section V-I, not of the models themselves.

On the model side, the v1→v5 progression (mAP_{50} 0.465 → 0.847) with a slightly positive validation-to-test gap of $+0.030$ at v5 points to dataset scale and diversity, rather than architectural changes, as the dominant lever on this corpus. We read this trajectory as an *existence proof of within-corpus scaling* for the deployed detector, not as a generalisable scal-

ing law. The 0.38-point headline jump sits well outside the ± 0.01–0.02 mAP run-to-run variance that YOLO fine-tuning typically reports at this scale, but the intermediate v2 waypoint and the per-class ordering have not been resolved under a repeated-seed, repeated-split protocol. The qualifications in Section III-A (single seed, targeted additions, same-corpus test split) bound how far this generalises. The COCO baseline of essentially 0.0002 mAP on these four classes confirms that a generic pretrained detector is unusable for domain-specific security labels. The confusion matrix is diagonally dominant; the largest inter-class cell is Person→Knife at 0.11, and the bulk of the remaining true-class mass falls into the background column rather than into another target class. A peak F1 confidence aligned with YOLO's default 0.25 threshold means the model ships without any post-deployment threshold tuning.

The INT8 HEF re-evaluated on the deployed Hailo-8L over the same test split reaches mAP@0.5 = 0.854 vs. 0.847 FP32 ($\Delta = +0.007$) and mAP@0.5:0.95 = 0.682 vs. 0.670 ($\Delta = +0.012$). Within this 622-image / 351-instance sample the post-training quantisation step contributes no measurable accuracy loss, and the slight positive delta is attributable to the deployment NMS thresholds compiled into the on-chip post-process (Section V-J). The 22.9 MB INT8 artefact also sustains 52.2 FPS at 18.4 ms on-chip latency, at 5.89 ± 1.43 W of measured Pi 5 + Hailo-8L board power under concurrent inference and CPU-side service load (Section V-J). We deliberately do not draw a head-to-head against a GPU reference, because the two devices sit in different thermal and PSU classes.

On the systems side, the load-bearing result is narrower than a platform claim. The asynchronous, bounded-queue, CPU-pinned pipeline keeps the NPU inference path isolated from the concurrent CPU-side services of Section IV-G, and that isolation is what keeps the 52.2 FPS line in Section V-I flat across operating modes, rather than dependent on which services happen to be quiet. The individual services are not the contribution. The contribution is the property that the inference frame rate does not depend on them.

The broader observation is narrower than a replacement claim. NPU-equipped single-board computers now have enough headroom that a sub-$500 device can host the full inference and service stack of a home surveillance platform without cloud offload, at real-time frame rates, given strict NPU/CPU separation and a domain-specific detector. Whether any such device replaces a specific commercial cloud-backed camera in practice is a separate empirical question, and it requires a continuous-deployment evaluation that this paper did not conduct.

Tightening the result into a deployment claim needs three lines of follow-up work. We group them here for compactness. The first is an in-situ end-to-end security evaluation: alert precision/recall and missed-event rate on unstaged multi-day video, nuisance-condition sweeps (lighting, multi-subject, occlusion, motion blur), physical replay-attack testing with screens, curved prints, and masks, fault injection on the tunnel and alert paths, a multi-week stability study, and a matched-stimulus comparison against a commercial cloud camera. The second is a generalisability re-analysis of the dataset trajectory: repeated seeds and repeated splits over v1→v2→v5, an inter-annotator-agreement study, an external home-surveillance benchmark, and an open-set evaluation of the MobileNetV2 verifier with household-clutter negatives that would turn its 99.0% in-distribution figure into a deployment-calibrated number. The third is a set of scoped-but-unevaluated component upgrades: a proper face anti-spoof stage calibrated on in-domain data, a broader class vocabulary, a temporal activity-recognition layer on top of the detector, and on-device TTS for Narada so the voice path keeps working through public-network outages.

APPENDIX A HARDWARE, SOFTWARE, AND MODEL SPECIFICATIONS

This appendix consolidates the hardware, software, and model specifications of the deployed system so that the results reported in Section V can be reproduced on comparable equipment. Only the deployed configuration is listed; development-time alternatives are not included.

A. HARDWARE

Table 12 lists the host board, accelerator, camera, storage, and network interface used for every reported measurement.

TABLE 12. Hardware Specification

Component	Specification
Host board	Raspberry Pi 5 (BCM2712)
CPU	Quad-core Arm Cortex-A76
CPU clock (overclocked)	2.8 GHz (`arm_freq=2800`)
RAM	16 GB LPDDR4X-4267
Storage	1 TB Crucial SATA SSD (USB 3.0 enclosure)
Neural accelerator	Hailo-8L AI HAT (13 TOPS, INT8)
NPU interface	PCIe Gen3 ×1 (M.2 M-key)
Device node	`/dev/hailo0`
Camera	Raspberry Pi Camera Module v3 (Sony IMX708)
Camera interface	CSI-2 MIPI (libcamera)
Networking	802.11ac Wi-Fi
Power supply	5 V / 5 A USB-C (official PSU)
Estimated BOM	US$411

B. SOFTWARE STACK

The software stack is summarised in Table 13. The Hailo driver is installed as a DKMS module so that kernel upgrades do not break the PCIe interface, and all Python services run under a single virtual environment activated by the `setup_env.sh` script shipped with the project.

C. MODELS

Three neural networks are deployed, all compiled to INT8 HEF files for the Hailo-8L. The primary detector is a YOLOv8s model fine-tuned on the four-class dataset (Section III), trained for 150 epochs, and quantised with the Hailo DFC v3.33.1. The classifier is a MobileNetV2 that labels

TABLE 13. Software Stack Specification

Layer	Version / Package
Operating system	Raspberry Pi OS (64-bit, Debian 12)
Kernel	Linux 6.12.x
Hailo driver	`hailo_pci` 4.20.0 (DKMS)
Hailo runtime	HailoRT 4.20.0
Hailo compiler	Hailo Dataflow Compiler v3.33.1
TAPPAS framework	3.28.x / 3.31.0
GStreamer	1.22
Python	3.11
Web framework	FastAPI + Uvicorn
ASGI media	aiortc (WebRTC), `python-multipart`
Speech	`SpeechRecognition`, `PyAudio`
LLM backend	Groq API, `gpt-oss-120b` (free tier)
Auth primitives	`hashlib.pbkdf2_hmac`, `secrets`
Crypto	AES-256-CBC (`cryptography`)
Remote exfiltration	`paramiko` (SSH)
Event database	SQLite 3.40
Public ingress	Cloudflare Tunnel (`cloudflared`)

person crops as Safe or Weapon. The third is MiDaS Small, used only as a coarse depth-variance pre-filter (not a face anti-spoof decision; Section V-L). Table 14 lists all deployed artefacts.

TABLE 14. Deployed Models and Artefacts (Hailo-8L)

Artefact	Size	Task	FPS	Latency
`best_v5.hef` (YOLOv8s)	22.9 MB	4-class detection	52.2	18.4 ms
`mobilenetv2_garuda.hef`	7.1 MB	Safe/Weapon crop cls.	500+	<2 ms
`midas_depth.hef` (MiDaS Small)	35 MB	Monocular depth	200+	<5 ms
`libyolo_hailortpp_post.so`	561 KB	Custom-label NMS	—	—

D. SERVICE-SIDE PARAMETERS

The per-engine implementation parameters referenced from Section IV-G are split into two tables for readability: the security-relevant primitives (Table 15) and the operational/scheduling values (Table 16). None of these values is load-bearing for any reported result; they are listed only for reproducibility.

TABLE 15. Security-relevant Primitives

Parameter	Value
PBKDF2 iterations	600,000 (SHA-256, per-user salt)
Session token	HTTP-only cookie, server-side lookup
Admin 2FA	6-digit OTP via SMTPS, 10-min expiry
Evidence-clip cipher	AES-256-CBC, key from env var
Evidence-clip transport	SSH (`paramiko`)
Privacy-mode blur kernel	Gaussian, k=51 px, σ=30 px
Public ingress	Cloudflare Tunnel

E. V6 FINE-TUNE: FULL CONFIGURATION AND PER-CLASS REGRESSION ANALYSIS

The compressed v6 discussion in Section V-E1 is expanded here for reproducibility. The v6 fine-tune used a fresh YOLOv8s initialisation trained on an expanded corpus

TABLE 16. Operational and Scheduling Parameters

Parameter	Value
Alert SMTPS cooldown (per label)	60 s
Mode-flag set	DND, email-off, idle, emergency, privacy
Schedule-monitor poll	30 s
ARP presence poll	30 s
Presence grace window	90 s
Deadman heartbeat check	10 s
Deadman silence threshold	180 s
FastAPI bind	`localhost:8080` (Uvicorn)
REST routes (approx.)	45
Live-video channels	MJPEG, WebSocket binary, WebRTC (aiortc)
Log rotation	7-day, append-only
Catch-up endpoint	`/api/events?since=T` (SQLite-backed)

of 4,720 primary-class Person images (13,014 Person instances), with an additional 1,903 Open Images Person photographs drawn under a different seed to raise crowded-scene diversity. The augmentation schedule was pushed beyond the v5 defaults to `copy_paste`= 0.5, `mixup`= 0.2, and `label_smoothing`= 0.05, with 200 training epochs and a later mosaic-close schedule tuned for crowded scenes.

The per-class outcome at the deployed confidence threshold was: overall test mAP@0.5 0.847 → 0.741; Person recall 0.609 → 0.481; Hammer recall 0.848 → 0.765; Knife recall 0.817 → 0.913 (the only class to improve); Scissors recall held near 0.921. Lowering the confidence threshold to 0.10 recovered overall mAP only to 0.766 and did not move Person recall, which rules out a pure threshold-tuning explanation. Two effects account for the regression: the additional 1,903 Open Images Person photographs shifted the training distribution away from the balanced v5 composition that the carefully-curated weapon classes depend on, and the aggressive copy-paste and mixup schedule over-regularised the classification head, degrading all classes that had already converged to a good optimum under v5. v5 was retained as Pareto-optimal on the test set because it keeps the alert-class recalls (Hammer 0.848, Knife 0.817, Scissors 0.929) intact while the v6 run gave up Hammer and overall mAP in exchange for the single Knife-recall gain. The v6 experiment is included as a documented negative result that motivates the four architectural directions flagged in the main text.

AUTHOR CONTRIBUTIONS

The work reported here was conceived and executed by the first author (G. V. Manikanta, student) under the academic supervision of the second author (S. Tangi), who served as the faculty advisor for this project at the Department of Electrical and Electronics Engineering, Manipal Institute of Technology. Using the Contributor Roles Taxonomy (CRediT): **G. V. Manikanta** — conceptualisation, methodology, software (all firmware, GStreamer pipeline, FastAPI web server, voice assistant, authentication and evidence-capture subsystems), hardware integration (Pi 5 + Hailo-8L + camera stack), dataset curation and re-annotation across v1/v2/v5, model

training and INT8 HEF compilation, all reported measurements, figure and table preparation, original draft writing, and revisions. **S. Tangi** — supervision and academic mentorship throughout the project, guidance on experimental scope and methodology, review and validation of the experimental protocol, and writing review and editing of the manuscript. Both authors read and approved the final manuscript.

CODE AND DATA AVAILABILITY

The complete codebase for the work reported here — Hailo GStreamer pipeline, FastAPI web server, voice assistant, authentication and evidence-capture subsystems, dataset-preparation scripts, and the evaluation artefacts referenced in Section V (INT8 mAP, open-set verifier test, power-measurement traces, and the 720 s FPS time-series used to produce Fig. 17) — is released under an open license at https://github.com/Manikanta25055/Garuda_26. The deployed HEF artefacts (`best_v5.hef` for the primary detector, `mobilenetv2_garuda.hef` for the secondary verifier, and `midas_depth.hef` for the depth-variance pre-filter) and the `libyolo_hailortpp_post.so` post-processing library are included in the same repository. The dataset split itself is not redistributed directly, but the SHA-256 manifest of image identifiers and the per-split label files are available alongside the code so that an equivalent split can be reconstructed from the upstream Roboflow Universe and OpenImages v7 sources (both CC BY 4.0) using the scripts in the repository.

ETHICS AND DATA-USE STATEMENT

No human-subjects study, no interaction with identifiable individuals, and no collection of original video or biometric data from third parties was undertaken for this work, and accordingly no Institutional Review Board (IRB) or ethics-committee approval was required. All training and evaluation imagery for the four-class detector was drawn from publicly released, permissively licensed corpora (Roboflow Universe ''Knives & Scissors Training v2'' and Open Images v7, both CC BY 4.0); the upstream providers are responsible for consent and rights clearance on those images, and no identities are inferred or reported in this paper. The CelebA-Spoof subset used for the short sanity check of Section V-L was accessed from the authors' publicly released research distribution under its stated non-commercial academic research terms; it is used only to bound the scope of the depth-variance pre-filter, and no identity, demographic, or attribute inference is performed or released. All operational imagery captured on the deployed Raspberry Pi Camera during development was recorded in the first author's private residence with the consent of the household; no such footage is redistributed with this paper. Evidence clips produced by the deployed system are encrypted at rest (AES-256-CBC) and remain on the user's own hardware; the system is designed so that raw video never leaves the device to a third-party service. Commercial product names and prices cited in Section VI are used for the sole purpose of cost-of-ownership comparison and do not constitute an endorsement.

ACKNOWLEDGMENT

The authors thank the Department of Electrical and Electronics Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, for providing the academic environment in which this work was carried out. The authors are grateful to the open-source communities behind the Hailo TAPPAS, Ultralytics YOLOv8, GStreamer, FastAPI, Raspberry Pi, and Open Images projects, on whose tooling and datasets this work directly depends. All hardware, cloud services, and network costs reported here were self-funded by the first author; no external funding was received.

Use of Generative AI

In accordance with IEEE policy on the use of generative AI tools, the authors disclose that Anthropic's Claude (Opus 4.7) was used in a limited editorial capacity during manuscript preparation — specifically, for light copy-editing of selected paragraphs, minor rephrasing to improve clarity and readability, and consistency checks on cross-references. All technical content, experimental design, measurements, source code, figures, and the scientific claims of this paper were conceived, produced, and verified by the authors, who take full responsibility for the manuscript.

REFERENCES

[1] Z. Xu, J. Li, and M. Zhang, ''A surveillance video real-time analysis system based on edge-cloud and FL-YOLO cooperation in coal mine,'' *IEEE Access*, vol. 9, pp. 161 498–161 512, 2021.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, ''You only look once: Unified, real-time object detection,'' in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[3] Hailo Technologies Ltd., ''Hailo-8L M.2 AI acceleration module datasheet,'' 2024. [Online]. Available: https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accel-for-ai-light-applications/

[4] G. Bueno *et al.*, ''Real-time edge computing vs. GPU-accelerated pipelines for low-cost microscopy applications,'' *Electronics*, vol. 14, no. 6, Art. no. 930, 2025.

[5] R. Al Amin, M. Hasan, V. Wiese, and R. Obermaisser, ''FPGA-based real-time object detection and classification system using YOLO for edge computing,'' *IEEE Access*, vol. 12, pp. 61 080–61 093, 2024.

[6] C. Dong, C. Pang, Z. Li, X. Zeng, and X. Hu, ''PG-YOLO: A novel lightweight object detection method for edge devices in industrial Internet of Things,'' *IEEE Access*, vol. 10, pp. 40 177–40 186, 2022.

[7] G. Jocher, A. Chaurasia, and J. Qiu, ''Ultralytics YOLOv8,'' 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] Hailo Technologies Ltd., ''TAPPAS — Hailo's AI application development framework,'' 2024. [Online]. Available: https://github.com/hailo-ai/tappas

[9] Raspberry Pi Ltd., ''Raspberry Pi 5 product brief,'' 2023. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5/

[10] S. Ramírez, ''FastAPI — Modern, fast web framework for building APIs with Python,'' 2019. [Online]. Available: https://fastapi.tiangolo.com

[11] GStreamer Team, ''GStreamer: Open source multimedia framework,'' 2024. [Online]. Available: https://gstreamer.freedesktop.org

[12] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, ''Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,'' *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.

[13] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, ''MobileNetV2: Inverted residuals and linear bottlenecks,'' in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.

[14] B. Kaliski, "PKCS #5: Password-based cryptography specification version 2.0," RFC 2898, Internet Eng. Task Force, Sep. 2000.
[15] Cloudflare, Inc., "Cloudflare Tunnel documentation," 2024. [Online]. Available: https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/
[16] M. T. Bhatti, M. G. Khan, M. Aslam, and M. J. Fiaz, "Weapon detection in real-time CCTV videos using deep learning," *IEEE Access*, vol. 9, pp. 34 366–34 382, 2021, doi: 10.1109/ACCESS.2021.3059170.
[17] A. Yadav, N. Gupta, and A. Sharma, "YOLOv7-DarkVision: Weapon detection in low-light surveillance videos with illumination-aware enhancement," *Digital Signal Processing*, vol. 145, Art. no. 104342, 2024, doi: 10.1016/j.dsp.2023.104342.
[18] A. Amado-Garfías, J. F. Martínez-Trinidad, J. A. Carrasco-Ochoa, and J. C. Ramírez-Cruz, "Enhancing armed-person detection in surveillance by combining YOLOv4, geometric heuristics, and classical machine-learning verifiers," *IEEE Access*, vol. 12, pp. 116 732–116 746, 2024, doi: 10.1109/ACCESS.2024.3442728.
[19] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, "Deep learning for face anti-spoofing: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5609–5631, May 2023, doi: 10.1109/TPAMI.2022.3215850.
[20] Y. Zhang, Z. Yin, Y. Li, G. Yin, J. Yan, J. Shao, and Z. Liu, "CelebA-Spoof: Large-scale face anti-spoofing dataset with rich annotations," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Glasgow, UK, 2020, pp. 70–85.
[21] Q. Zhou, K.-Y. Zhang, T. Yao, X. Lu, S. Ding, and L. Ma, "Test-time domain generalization for face anti-spoofing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, 2024, pp. 175–185.
[22] B. Blackshear, "Frigate — NVR with realtime local object detection for IP cameras," 2024. [Online]. Available: https://github.com/blakeblackshear/frigate
[23] H. Alqahtani, M. A. Cheema, and A. N. Toosi, "Benchmarking deep learning models for object detection on edge computing devices," *arXiv preprint arXiv:2409.16808*, 2024.
[24] D. Rey, A. M. Bernardos, P. Dobrzycki, A. Carramiñana, L. Bergesio, J. A. Besada, and J. R. Casar, "A performance analysis of YOLOv8 on edge devices for aerial imagery," *arXiv preprint arXiv:2502.15737*, 2025.
[25] W. Ullah, T. Hussain, F. U. M. Ullah, M. Y. Lee, and S. W. Baik, "TransCNN: Hybrid CNN and transformer mechanism for surveillance anomaly detection," *IEEE Trans. Ind. Informat.*, 2022, doi: 10.1109/TII.2021.3094920.
[26] E. I. R. Jacob *et al.*, "An enhanced surveillance system for gun and knife detection using YOLOv8 and Raspberry Pi," in *Proc. IEEE Int. Conf. Communication, Computing and Internet of Things (IC3IoT)*, Chennai, India, 2024, pp. 1–6, doi: 10.1109/IC3IoT60841.2024.10550256.
[27] M. B. Musthafa, S. Huda, T. T. Nguyen, Y. Kodera, and Y. Nogami, "Optimized ensemble deep learning for real-time intrusion detection on resource-constrained Raspberry Pi devices," *IEEE Access*, vol. 13, pp. 113 544–113 556, 2025, doi: 10.1109/ACCESS.2025.3584373.
[28] L. Sumi and S. Dey, "YOLOv5-based weapon detection systems with data augmentation," *Int. J. Computers and Applications* (Taylor & Francis), vol. 45, no. 4, pp. 288–296, 2023, doi: 10.1080/1206212X.2023.2182966.
[29] O. Taiwo, A. E. Ezugwu, O. N. Oyelade, and M. S. Almutairi, "Enhanced intelligent smart home control and security system based on deep learning model," *Wireless Commun. Mobile Comput.* (Wiley/Hindawi), vol. 2022, Art. no. 9307961, 2022, doi: 10.1155/2022/9307961.
[30] N. Hnoohom, P. Chotivatunyu, and A. Jitpattanakul, "ACF: An armed CCTV footage dataset for enhancing weapon detection," *Sensors (MDPI)*, vol. 22, no. 19, Art. no. 7158, 2022, doi: 10.3390/s22197158.
[31] Ring LLC, "Stick Up Cam Battery," product page, 2026. [Online]. Available: https://ring.com/products/stick-up-cam-battery
[32] Ring LLC, "Ring Protect plans," 2026. [Online]. Available: https://ring.com/protect-plans
[33] Google LLC, "Nest Cam (battery)," Google Store product page, 2026. [Online]. Available: https://store.google.com/us/product/nest_cam_battery
[34] Google LLC, "Google Home Premium (formerly Nest Aware)," Google Store, 2026. [Online]. Available: https://store.google.com/us/product/nest_aware
[35] SimpliSafe, Inc., "Outdoor Security Camera Bundle," product page, 2026. [Online]. Available: https://simplisafe.com/outdoor-security-camera-bundle
[36] SimpliSafe, Inc., "Monitoring plans," support page, 2026. [Online]. Available: https://support.simplisafe.com/page/monitoring-plans
[37] Anker Innovations Ltd., "eufyCam S3 Pro 2-Cam Kit," eufy product page, 2026. [Online]. Available: https://www.eufy.com/products/t88921w1
[38] Reolink Digital Technology Co., Ltd., "Argus 4 Pro," product page, 2026. [Online]. Available: https://reolink.com/us/product/argus-4-pro/